

DOCUMENTO TÉCNICO DE DEFENSA ORAL – PARTE 1

Sistema de Mesa de Ayuda · Cooperativa Comunicarlos

Materia: Paradigmas de Programación V

Stack: Python 3.11+ · FastAPI · MongoDB · Bootstrap 5.3 SPA

Tests: 111 unitarios (82 dominio + 29 servicios)

Patrones: State, Strategy, Factory, Observer, Repository, Service Layer, Singleton

PARTE 1 – ANÁLISIS ARCHIVO POR ARCHIVO

1.1 Entrada de la aplicación

`app/main.py`

Punto de entrada de la aplicación FastAPI. Responsabilidad exclusiva de **composición**: ensambla la aplicación registrando routers, middleware CORS y archivos estáticos del frontend.

Responsabilidades concretas:

- Instancia `FastAPI` con metadata para documentación OpenAPI (`/docs` , `/redoc`).
- Registra `CORSMiddleware` con política permisiva (`allow_origins=["*"]`).
- Monta 6 routers: `health_router` , `auth_router` , `usuarios_router` , `requerimientos_router` , `notificaciones_router` , `servicios_router` .
- Monta archivos estáticos del frontend SPA en `/app` , documentación Sphinx en `/documentacion` y diagramas UML en `/uml` .
- Define `@app.on_event("startup")` con `_bootstrap_supervisor()` : crea un Supervisor inicial (`admin@comunicarlos.com.ar`) si no existe, y auto-monitorea a todos los operadores y técnicos existentes.
- Define ruta raíz `/` que sirve `static/index.html` .

Decisión de diseño: `main.py` no contiene lógica de negocio. Su única razón de cambio sería agregar un router nuevo o modificar middleware. Esto respeta el **Principio de Responsabilidad Única (SRP)**.

app/dependencies.py

Implementa **inyección de dependencias manual**. Es el único punto donde se instancian repositorios y servicios concretos. Funciona como **Composition Root**.

Configuración condicional:

```
if settings.USE_MONGO:
    # Repositorios MongoDB (producción)
    _requerimiento_repo = RequerimientoRepositoryMongo(get_collection("re
    _usuario_repo       = UsuarioRepositoryMongo(get_collection("usuarios
    _notificacion_repo  = NotificacionRepositoryMongo(get_collection("not
else:
    # Repositorios in-memory (tests/desarrollo)
    _requerimiento_repo = RequerimientoRepositoryInMemory()
    _usuario_repo       = UsuarioRepositoryInMemory()
    _notificacion_repo  = NotificacionRepositoryInMemory()
```

Observer Pattern wiring:

```
_event_bus = EventBus()

def _on_notificacion_creada(notificacion):
    _notificacion_repo.guardar(notificacion)

_notificacion_listener = NotificacionListener(on_notificacion_creada=_on_
_event_bus.suscribir(_notificacion_listener)
```

Funciones getter de servicios (cada una inyecta repos):

Función	Service que retorna	Repos inyectados
<code>get crear requerimiento_service()</code>	<code>CrearRequerimientoService</code>	<code>requerimiento_repo</code> , <code>usuario_repo</code> , <code>factory</code> , <code>notificacion_service</code>
<code>get asignar_tecnico_service()</code>	<code>AsignarTecnicoService</code>	<code>requerimiento_repo</code> , <code>usuario_repo</code> , <code>notificacion_service</code>
<code>get derivar requerimiento_service()</code>	<code>DerivarRequerimientoService</code>	<code>requerimiento_repo</code> , <code>usuario_repo</code> , <code>notificacion_service</code>

Función	Service que retorna	Repos inyectados
<code>get_comentario_service()</code>	<code>ComentarioService</code>	<code>requerimiento_repo, usuario_repo, notificacion_service</code>
<code>get_resolver_requerimiento_service()</code>	<code>ResolverRequerimientoService</code>	<code>requerimiento_repo, usuario_repo, notificacion_service</code>
<code>get_reabrir_requerimiento_service()</code>	<code>ReabrirRequerimientoService</code>	<code>requerimiento_repo, usuario_repo, notificacion_service</code>
<code>get_notificacion_service()</code>	<code>NotificacionService</code>	<code>usuario_repo, notificacion_repo</code>

Decisión de diseño: Si mañana se cambia de MongoDB a PostgreSQL, solo se modifica este archivo. Los servicios y routers no se enteran porque dependen de interfaces abstractas

(`IUsuarioRepository` , `IRequerimientoRepository`), no de implementaciones concretas.

Esto es **Dependency Inversion Principle (DIP)**.

1.2 Capa de Autenticación (`app/auth/`)

`app/auth/security.py`

Dos funciones puras (wrappers de passlib con bcrypt):

```
_pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return _pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return _pwd_context.verify(plain_password, hashed_password)
```

¿Por qué no es una clase? Son funciones sin estado. No hay instancia que mantener. Es convención estándar en Python para wrappers stateless.

`app/auth/jwt_handler.py`

Dos funciones (wrappers de python-jose):

```
def create_access_token(user_id: UUID, email: str, rol: str,
                        expires_delta: Optional[timedelta] = None) -> str
```

```

expire = datetime.now(timezone.utc) + (
    expires_delta or timedelta(minutes=settings.JWT_EXPIRE_MINUTES)
)
payload = {"sub": str(user_id), "email": email, "rol": rol, "exp": expire}
return jwt.encode(payload, settings.JWT_SECRET_KEY, algorithm=settings.JWT_ALGORITHM)

def decode_access_token(token: str) -> dict:
    return jwt.decode(token, settings.JWT_SECRET_KEY, algorithms=[settings.JWT_ALGORITHM])

```

Decisión de diseño: El `rol` en el JWT se deriva de `usuario.__class__.__name__` (Solicitante, Operador, Tecnico, Supervisor). No se almacena un campo `rol` separado en la entidad. Se aprovecha el polimorfismo del dominio.

app/auth/auth_dependencies.py

Dos dependencias reutilizables de FastAPI:

`get_current_user(credentials)` : Extrae token Bearer del header `Authorization` , lo decodifica con `decode_access_token` , busca el usuario en el repositorio por UUID y lo retorna como entidad de dominio. Si el token es inválido o el usuario no existe → `HTTPException 401` .

```

def get_current_user(credentials: HTTPAuthorizationCredentials = Depends(Header(
    try:
        payload = decode_access_token(credentials.credentials)
        user_id = UUID(payload["sub"])
    except (JWTError, KeyError, ValueError):
        raise HTTPException(status_code=401, detail="Token invalido o expirado",
                            headers={"WWW-Authenticate": "Bearer"})
    repo = get_usuario_repo()
    usuario = repo.obtener_por_id(user_id)
    if usuario is None:
        raise HTTPException(status_code=401, detail="Usuario no encontrado",
                            headers={"WWW-Authenticate": "Bearer"})
    return usuario

```

`require_roles(allowed_roles: List[str])` : Función que retorna una dependencia. Esta dependencia invoca internamente a `get_current_user` y luego verifica que `usuario.__class__.__name__` esté en `allowed_roles` . Si no → `HTTPException 403` .

```

def require_roles(allowed_roles: List[str]):
    def _check_role(usuario=Depends(get_current_user)):

```

```

    rol = usuario.__class__.__name__
    if rol not in allowed_roles:
        raise HTTPException(status_code=403,
                             detail=f"Rol '{rol}' no tiene permiso para")

    return usuario
return _check_role

```

¿Por qué **require_roles** es una función que retorna una función? Porque FastAPI requiere que las dependencias sean callables. Para parametrizar los roles permitidos, se usa un closure. En el router queda declarativo:

```

usuario = Depends(require_roles(["Operador", "Supervisor"]))

```

1.3 Capa de Dominio (**app/domain/**)

app/domain/entities/usuario.py – Clase base abstracta

```

@dataclass
class Usuario(ABC):
    id: UUID
    nombre: str
    email: str
    password_hash: str
    creado_en: datetime
    ultimo_acceso: datetime

    def __post_init__(self) -> None:
        self._validar_email()
        self._validar_nombre()

    @abstractmethod
    def puede_ver(self, req: Requerimiento) -> bool: pass

    @abstractmethod
    def puede_comentar(self, req: Requerimiento) -> bool: pass

    def actualizar_ultimo_acceso(self, cuando=None) -> None:
        self.ultimo_acceso = cuando or datetime.now(timezone.utc)

```

- Validaciones internas: `_validar_email()` (verifica @ en el email), `_validar_nombre()` (no vacío).
- Métodos abstractos `puede_ver()` y `puede_comentar()` obligan a cada subclase a definir sus reglas de permisos específicas.

¿Por qué es ABC? Porque nunca se instancia un "Usuario genérico". Siempre es un Solicitante, Operador, Técnico o Supervisor. Los métodos abstractos obligan a cada subclase a definir sus reglas de permisos.

app/domain/entities/solicitante.py

```
@dataclass
class Solicitante(Usuario):
    suscripciones: List[Suscripcion] = field(default_factory=list)

    def puede_ver(self, req) -> bool:
        return req.autor_id == self.id                # solo sus propios requerimientos

    def puede_comentar(self, req) -> bool:
        return req.autor_id == self.id                # solo en sus propios requerimientos

    def esta_suscripto(self, servicio: Servicio) -> bool:
        return any(s.servicio.id == servicio.id for s in self.suscripciones)

    def agregar_suscripcion(self, servicio: Servicio) -> Suscripcion:
        if self.esta_suscripto(servicio):
            raise DomainRuleError(f"Ya esta suscripto al servicio {servicio}")
        suscripcion = Suscripcion(id=uuid4(), solicitante_id=self.id,
                                   servicio=servicio, desde=datetime.now())
        self.suscripciones.append(suscripcion)
        return suscripcion

    def comentar(self, req, cuerpo: str) -> None:
        if not self.puede_comentar(req):
            raise DomainRuleError("No puede comentar este requerimiento")
        req.agregar_comentario(autor_id=self.id, autor_nombre=self.nombre)
```

app/domain/entities/operador.py

```

@dataclass
class Operador(Usuario):
    def __post_init__(self):
        super().__post_init__()
        self._validar_email_corporativo()

    def _validar_email_corporativo(self):
        if not self.email.endswith("@comunicarlos.com.ar"):
            raise ValueError("Operadores deben usar email corporativo @cc

    def puede_ver(self, req) -> bool:    return True    # ve todos
    def puede_comentar(self, req) -> bool: return True    # comenta en todo

    def asignar(self, req, tecnico) -> None:
        req.asignar(tecnico_id=tecnico.id, tecnico_nombre=tecnico.nombre,
                    asignado_por_id=self.id, asignado_por_nombre=self.nombre

    def reasignar(self, req, tecnico) -> None:
        self.asignar(req, tecnico)    # internamente cierra la previa y

    def reabrir(self, req, motivo: str) -> None:
        req.reabrir(actor_id=self.id, actor_nombre=self.nombre, motivo=motivo

    def comentar(self, req, cuerpo: str) -> None:
        req.agregar_comentario(actor_id=self.id, autor_nombre=self.nombre

```

Nota: Valida email corporativo `@comunicarlos.com.ar` en el constructor. Esto es validación de dominio que se ejecuta al instanciar la entidad.

app/domain/entities/tecnico.py

```

@dataclass
class Tecnico(Usuario):
    especialidades: List[Servicio] = field(default_factory=list)

    def __post_init__(self):
        super().__post_init__()
        self._validar_email_corporativo()

    def puede_ver(self, req) -> bool:
        return req.asignado_a_id() == self.id    # solo requerimientos

```

```

def puede_comentar(self, req) -> bool:
    return req.asignado_a_id() == self.id          # solo en requerimier

def tiene_especialidad(self, servicio: Servicio) -> bool:
    return any(e.id == servicio.id for e in self.especialidades)

def resolver(self, req) -> None:
    if not self.puede_ver(req):
        raise DomainRuleError("No puede resolver un requerimiento no a
    req.resolver(tecnico_id=self.id, tecnico_nombre=self.nombre)

def derivar(self, req, destino: Tecnico, motivo: str,
             actor_id=None, actor_nombre=None) -> None:
    if not self.puede_ver(req):
        raise DomainRuleError("No puede derivar un requerimiento no a
    req.derivar(origen_id=self.id, origen_nombre=self.nombre,
                 destino_id=destino.id, destino_nombre=destino.nombre,
                 motivo=motivo, actor_id=actor_id, actor_nombre=actor_

```

app/domain/entities/supervisor.py

```

@dataclass
class Supervisor(Usuario):
    monitoreados_ids: List[UUID] = field(default_factory=list)
    _notificaciones: List[Notificacion] = field(default_factory=list)

    def puede_ver(self, req) -> bool:          return True    # ve todos
    def puede_comentar(self, req) -> bool:    return False    # NO puede come

    def monitorear(self, usuario_id: UUID) -> None:
        if usuario_id not in self.monitoreados_ids:
            self.monitoreados_ids.append(usuario_id)

    def esta_monitoreando(self, usuario_id: UUID) -> bool:
        return usuario_id in self.monitoreados_ids

    def recibir_notificacion(self, notificacion: Notificacion) -> None:
        self._notificaciones.append(notificacion)

    def notificaciones_no_leidas(self) -> List[Notificacion]:
        return [n for n in self._notificaciones if not n.leida]

```


Regla de negocio clave: `puede_comentar()` retorna `False`. El Supervisor supervisa, no opera ni comenta. Esto está en el dominio, no en el router.

app/domain/entities/requerimiento.py – State Pattern

Clase base con atributos privados protegidos por properties:

```
@dataclass
class Requerimiento:
    id: UUID
    servicio: Servicio
    titulo: str
    descripcion: str
    autor_id: UUID
    autor_nombre: str
    creado_en: datetime
    estado: EstadoRequerimiento = field(default_factory=EstadoAbierto)
    numero: int = 0
    resuelto_en: Optional[datetime] = None
    asignacion_actual: Optional[Asignacion] = None
    _historial_asignaciones: List[Asignacion] = field(default_factory=list)
    _comentarios: List[Comentario] = field(default_factory=list)
    _eventos: List[Evento] = field(default_factory=list)
```

Properties para acceso inmutable:

```
@property
def historial_asignaciones(self) -> tuple: return tuple(self._historial_asignaciones)
@property
def comentarios(self) -> tuple: return tuple(self._comentarios)
@property
def eventos(self) -> tuple: return tuple(self._eventos)
```

Las listas internas (`_historial_asignaciones`, `_comentarios`, `_eventos`) se exponen como **tuplas inmutables** vía properties. Nadie desde afuera puede hacer `req.comentarios.append(...)` – solo puede usar `req.agregar_comentario(...)` que valida y registra evento.

Transiciones de estado (State Pattern):

```

def asignar(self, tecnico_id, tecnico_nombre, asignado_por_id, asignado_por_nombre):
    if not self.estado.permite_asignar():
        raise DomainRuleError(f"El estado {self.estado.nombre()} no permite asignar")
    self._cerrar_asignacion_actual()
    nueva = Asignacion(tecnico_id=tecnico_id, tecnico_nombre=tecnico_nombre,
                        asignado_por_id=asignado_por_id, ...)
    self.asignacion_actual = nueva
    self._historial_asignaciones.append(nueva)
    self.estado = EstadoAsignado() # ← CAMBIO DE ESTADO
    self._registrar_evento(tipo="ASIGNACION", ...)

def resolver(self, tecnico_id, tecnico_nombre):
    if not self.estado.permite_resolver():
        raise DomainRuleError(f"El estado {self.estado.nombre()} no permite resolver")
    if self.asignacion_actual is None or self.asignacion_actual.tecnico_id != tecnico_id:
        raise DomainRuleError("Solo el tecnico asignado puede resolver")
    self.resuelto_en = datetime.now(timezone.utc)
    self.estado = EstadoResuelto() # ← CAMBIO DE ESTADO
    self._registrar_evento(tipo="RESOLUCION", ...)

def reabrir(self, actor_id, actor_nombre, motivo):
    if not self.estado.permite_reabrir():
        raise DomainRuleError(f"El estado {self.estado.nombre()} no permite reabrir")
    self.resuelto_en = None
    self.estado = EstadoReabierto() # ← CAMBIO DE ESTADO
    self._registrar_evento(tipo="REAPERTURA", detalle=f"Reabierto. Motivo: {motivo}")

def derivar(self, origen_id, origen_nombre, destino_id, destino_nombre, motivo):
    if self.asignacion_actual is None:
        raise DomainRuleError("No se puede derivar sin asignacion actual")
    if self.asignacion_actual.tecnico_id != origen_id:
        raise DomainRuleError("Solo el tecnico asignado puede derivar")
    if origen_id == destino_id:
        raise DomainRuleError("No se puede derivar al mismo tecnico")
    self._cerrar_asignacion_actual()
    nueva = Asignacion(tecnico_id=destino_id, ...)
    self.asignacion_actual = nueva
    self._historial_asignaciones.append(nueva)
    self._registrar_evento(tipo="DERIVACION", ...)

```

```

@dataclass
class Incidente(Requerimiento):
    urgencia: Urgencia = None
    categoria: CategoriaIncidente = None

    def __post_init__(self):
        if self.urgencia is None: raise ValueError("Incidente requiere ur
        if self.categoria is None: raise ValueError("Incidente requiere c
        super().__post_init__() # llama validar_titulo/descripcion y re

```

app/domain/entities/solicitud.py

```

@dataclass
class Solicitud(Requerimiento):
    categoria: CategoriaSolicitud = None

    def __post_init__(self):
        if self.categoria is None: raise ValueError("Solicitud requiere c
        super().__post_init__()

```

¿Por qué separar Incidente de Solicitud? Porque tienen atributos diferentes: Incidente tiene `urgencia` y `CategoriaIncidente`, Solicitud tiene `CategoriaSolicitud` sin urgencia. Las reglas de creación son diferentes. La herencia permite compartir el ciclo de vida (estados, asignaciones, comentarios).

app/domain/entities/estado_requerimiento.py – State Pattern

4 clases con `@dataclass(frozen=True, slots=True)`:

```

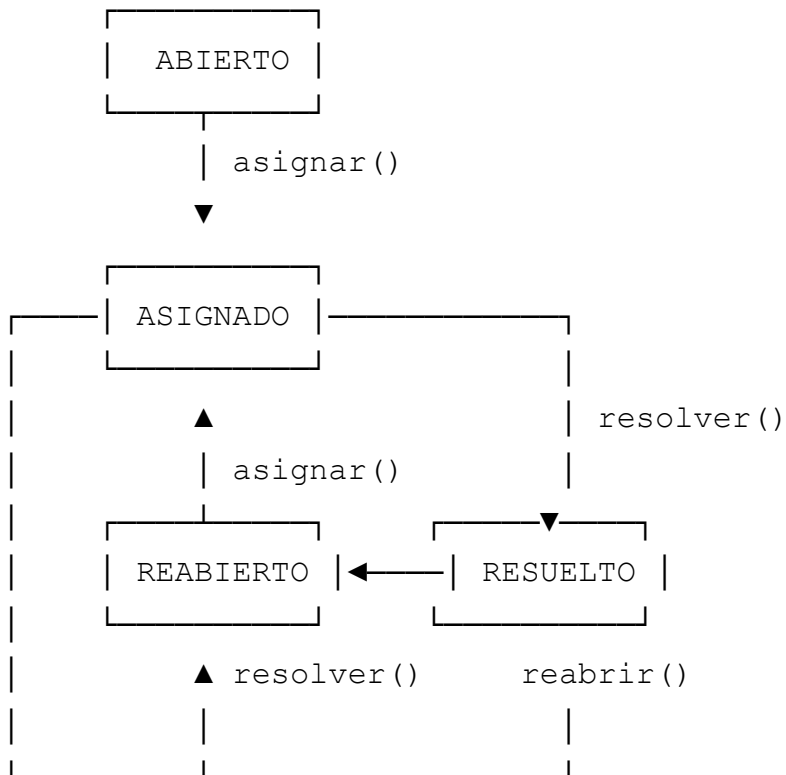
@dataclass(frozen=True, slots=True)
class EstadoRequerimiento(ABC):
    @abstractmethod
    def permite_asignar(self) -> bool: pass
    @abstractmethod
    def permite_resolver(self) -> bool: pass
    @abstractmethod
    def permite_reabrir(self) -> bool: pass

```

```
@abstractmethod
def nombre(self) -> str: pass
```

Clase	permite asignar()	permite resolver()	permite reabrir()	nombre()
EstadoAbierto	✓	✗	✗	"ABIERTO"
EstadoAsignado	✗ (usar derivar)	✓	✗	"ASIGNADO"
EstadoResuelto	✗	✗	✓	"RESUELTO"
EstadoReabierto	✓	✓	✗	"REABIERTO"

Diagrama de transiciones:



Nota importante: `EstadoAsignado.permite_asignar()` retorna `False`. Para cambiar técnico cuando ya está asignado se usa `derivar()`, no `asignar()`. `EstadoReabierto` permite tanto `asignar()` como `resolver()`.

¿Qué gana vs if/elif con strings?

Aspecto	Con strings/if	Con State Pattern
Agregar estado nuevo	Buscar TODOS los if/elif	Crear UNA clase nueva

Aspecto	Con strings/if	Con State Pattern
Testear transiciones	Mock de string global	Test unitario de cada estado
Leer qué permite un estado	Buscar condicionales	Mirar la clase directamente
Riesgo de bug	Alto (olvidar un case)	Bajo (ABC obliga a implementar)

app/domain/entities/asignacion.py

```
@dataclass
class Asignacion:
    tecnico_id: UUID
    tecnico_nombre: str
    asignado_por_id: UUID
    asignado_por_nombre: str
    desde: datetime
    hasta: Optional[datetime] = None

    def cerrar(self, cuando=None) -> None:
        if self.hasta is None:
            self.hasta = cuando or datetime.now(timezone.utc)

    def esta_activa(self) -> bool:
        return self.hasta is None
```

app/domain/entities/comentario.py – Frozen

```
@dataclass(frozen=True, slots=True)
class Comentario:
    autor_id: UUID
    autor_nombre: str
    cuerpo: str
    fecha: datetime

    def __post_init__(self):
        if not self.cuerpo or not self.cuerpo.strip():
            raise ValueError("El cuerpo del comentario no puede estar vacio")
```

frozen=True : La instancia es inmutable. `comentario.cuerpo = "otro"` lanza

`FrozenInstanceError` . Un comentario una vez creado no se edita — es un registro histórico.

app/domain/entities/evento.py – Frozen

```
@dataclass(frozen=True, slots=True)
class Evento:
    tipo: str          # CREACION, ASIGNACION, DERIVACION, COMENTARIO, RESC
    actor_id: UUID
    actor_nombre: str
    fecha: datetime
    detalle: str
```

Un evento es un **hecho histórico inmutable**. Es la traza de auditoría del requerimiento.

app/domain/entities/notificacion.py

```
@dataclass
class Notificacion:
    id: UUID
    destinatario_id: UUID
    empleado_id: UUID
    empleado_nombre: str
    requerimiento_id: UUID
    mensaje: str
    fecha: datetime
    leida: bool = False

    def marcar_leida(self):
        self.leida = True
```

app/domain/entities/suscripcion.py – Frozen

```
@dataclass(frozen=True, slots=True)
class Suscripcion:
    id: UUID
    solicitante_id: UUID
    servicio: Servicio
    desde: datetime
    hasta: Optional[datetime] = None
```

1.4 Value Objects (`app/domain/value_objects/`)

`app/domain/value_objects/servicio.py`

```
CATALOGO_SERVICIOS = {
    "telefonía": "a1b2c3d4-1111-1111-1111-000000000001",
    "internet":  "a1b2c3d4-1111-1111-1111-000000000002",
    "tv":        "a1b2c3d4-1111-1111-1111-000000000003",
}

NOMBRES_VALIDOS_SERVICIO = frozenset(CATALOGO_SERVICIOS.keys())

@dataclass(frozen=True, slots=True)
class Servicio:
    id: UUID
    nombre: str

    def __post_init__(self):
        if not self.nombre or not self.nombre.strip():
            raise ValueError("Servicio.nombre no puede estar vacío")
        if self.nombre.lower() not in NOMBRES_VALIDOS_SERVICIO:
            raise ValueError(f"Servicio.nombre debe ser uno de: {sorted(NOMBRES_VALIDOS_SERVICIO)}")

    def obtener_servicio(nombre: str) -> Servicio:
        nombre_lower = nombre.lower().strip()
        if nombre_lower not in CATALOGO_SERVICIOS:
            raise ValueError(f"Servicio '{nombre}' no válido. Disponibles: {sorted(NOMBRES_VALIDOS_SERVICIO)}")
        return Servicio(id=UUID(CATALOGO_SERVICIOS[nombre_lower]), nombre=nombre_lower)

    def listar_servicios() -> list[Servicio]:
        return [Servicio(id=UUID(uid), nombre=nombre) for nombre, uid in CATALOGO_SERVICIOS.items()]
```

El catálogo define 3 servicios con UUIDs fijos. `obtener_servicio()` valida y retorna el Value Object. `listar_servicios()` los lista todos.

`app/domain/value_objects/urgencia.py`

Jerarquía polimórfica con 3 niveles:

```
@dataclass(frozen=True, slots=True)
class Urgencia(ABC):
    @abstractmethod
```

```
def prioridad(self) -> int: pass
@abstractmethod
def nombre(self) -> str: pass
```

```
class UrgenciaCritica(Urgencia):    prioridad=3, nombre="critica"
class UrgenciaImportante(Urgencia): prioridad=2, nombre="importante"
class UrgenciaMenor(Urgencia):     prioridad=1, nombre="menor"
```

app/domain/value_objects/categoria_incidente.py

3 categorías con compatibilidad de servicio:

```
@dataclass(frozen=True, slots=True)
class CategoriaIncidente(ABC):
    @abstractmethod
    def nombre(self) -> str: pass
    @abstractmethod
    def servicios_compatibles(self) -> Set[str]: pass

    def es_compatible_con(self, servicio_nombre: str) -> bool:
        return servicio_nombre.lower() in self.servicios_compatibles()
```

Clase	nombre()	servicios_compatibles()
ServicioInaccesible	"servicio_inaccesible"	{"telefonía", "internet", "tv"}
BloqueoSIM	"bloqueo_sim"	{"telefonía"} – solo telefonía
PerdidaEquipo	"perdida_equipo"	{"telefonía", "internet", "tv"}

app/domain/value_objects/categoria_solicitud.py

```
class AltaServicio(CategoriaSolicitud): nombre = "alta_servicio"
class BajaServicio(CategoriaSolicitud): nombre = "baja_servicio"
```

1.5 Strategies (app/domain/strategies/)

app/domain/strategies/estrategia_asignacion.py –

Interfaz

```
class EstrategiaAsignacion(ABC):
    @abstractmethod
    def elegir_tecnico(self, req: Requerimiento, candidatos: List[Tecnico]):
```

app/domain/strategies/estrategia_por_especialidad.py

```
@dataclass
class EstrategiaPorEspecialidad(EstrategiaAsignacion):
    def elegir_tecnico(self, req, candidatos):
        if not candidatos:
            raise DomainRuleError("No hay tecnicos candidatos")
        for tecnico in candidatos:
            if tecnico.tiene_especialidad(req.servicio):
                return tecnico
        raise DomainRuleError(f"No hay tecnicos con especialidad en {req.servicio}")
```

Filtra candidatos que tengan el servicio del requerimiento en sus especialidades. Retorna el primero que coincida.

app/domain/strategies/estrategia_por_carga.py

```
@dataclass
class EstrategiaPorCarga(EstrategiaAsignacion):
    obtener_carga: Callable[[Tecnico], int]

    def elegir_tecnico(self, req, candidatos):
        if not candidatos:
            raise DomainRuleError("No hay tecnicos candidatos")
        return min(candidatos, key=self.obtener_carga)
```

Recibe una función `obtener_carga` por constructor que cuenta cuántos requerimientos activos tiene cada técnico. Elige el de menor carga con `min(candidatos, key=obtener_carga)`.

¿Por qué Strategy? El algoritmo de asignación podría cambiar. Hoy se asigna por especialidad o por carga. Mañana podría agregarse "por proximidad geográfica" sin modificar el código existente – solo se agrega una nueva clase. **Open/Closed Principle (OCP)**.

1.6 Factory (app/domain/factories/)

app/domain/factories/requerimiento_factory.py

```
class RequerimientoFactory:
    def crear_incidente(self, autor: Solicitante, servicio: Servicio,
                        titulo: str, descripcion: str,
                        urgencia: Urgencia, categoria: CategoriaIncidente):
        return Incidente(
            id=uuid4(), servicio=servicio, titulo=titulo,
            descripcion=descripcion, autor_id=autor.id,
            autor_nombre=autor.nombre, creado_en=datetime.now(timezone.utc),
            estado=EstadoAbierto(), urgencia=urgencia, categoria=categoria
        )

    def crear_solicitud(self, autor: Solicitante, servicio: Servicio,
                      titulo: str, descripcion: str,
                      categoria: CategoriaSolicitud) -> Solicitud:
        return Solicitud(
            id=uuid4(), servicio=servicio, titulo=titulo,
            descripcion=descripcion, autor_id=autor.id,
            autor_nombre=autor.nombre, creado_en=datetime.now(timezone.utc),
            estado=EstadoAbierto(), categoria=categoria,
        )
```

Qué hace internamente:

1. Genera un `UUID` para el nuevo requerimiento.
2. Establece `EstadoAbierto()` como estado inicial.
3. El `__post_init__()` del `Requerimiento` crea un `Evento(tipo="CREACION", ...)` automáticamente.
4. Retorna la entidad completa lista para persistir.

¿Por qué Factory? La creación de un requerimiento requiere coordinar UUID, estado, validación de campos y evento de creación. La Factory centraliza y estandariza la creación. Si se agrega un nuevo tipo (ej: `Consulta`), se agrega un método al factory sin modificar los existentes.

1.7 Events / Observer Pattern

(app/domain/events/)

app/domain/events/evento_dominio.py

```
@dataclass(frozen=True)
class EventoDominio:
    tipo: str
    datos: Dict[str, Any]
    fecha: datetime
```

app/domain/events/domain_event_listener.py

```
class DomainEventListener(ABC):
    @abstractmethod
    def handle(self, evento: EventoDominio) -> None: pass
```

app/domain/events/event_bus.py

```
@dataclass
class EventBus:
    _listeners: List[DomainEventListener] = field(default_factory=list)

    def suscribir(self, listener: DomainEventListener) -> None:
        self._listeners.append(listener)

    def publicar(self, evento: EventoDominio) -> None:
        for listener in self._listeners:
            listener.handle(evento)
```

app/domain/events/notificacion_listener.py

```
@dataclass
class NotificacionListener(DomainEventListener):
    on_notificacion_creada: Callable[[Notificacion], None]
```

```
def handle(self, evento: EventoDominio) -> None:
    dest_id = evento.datos.get("destinatario_id", evento.datos.get("s
    notificacion = Notificacion(
        id=uuid4(), destinatario_id=dest_id,
        empleado_id=evento.datos["empleado_id"],
        empleado_nombre=evento.datos.get("empleado_nombre", ""),
        requerimiento_id=evento.datos["requerimiento_id"],
        mensaje=evento.datos["mensaje"],
        fecha=datetime.now(timezone.utc), leida=False,
    )
    self.on_notificacion_creada(notificacion)
```

Wiring en `dependencies.py`:

```
_event_bus = EventBus()
_notificacion_listener = NotificacionListener(on_notificacion_creada=lamb
_event_bus.suscribir(_notificacion_listener)
```

El Observer Pattern está implementado y wired. Además, los servicios usan `NotificacionService` directamente para crear notificaciones (enfoque de capa de servicio). Ambos mecanismos coexisten — el `EventBus` está disponible para extensiones futuras y el `NotificacionService` maneja el flujo principal.

1.8 Excepciones de Dominio (`app/domain/exceptions/`)

```
class DomainError(Exception):      pass # Base
class DomainRuleError(DomainError): pass # Regla de negocio violada →
class PermissionDeniedError(DomainError): pass # Sin permisos → HT
class ValidationError(DomainError): pass # Datos inválidos → F
```

Decisión de diseño: Las excepciones son del dominio, no de HTTP. El dominio no sabe qué es un código 400 o 403. El router captura `DomainRuleError` y la traduce a `HTTPException(status_code=400)` . Esto mantiene la separación de capas.

1.9 Servicios (`app/services/`)

Cada servicio recibe repositorios y `notificacion_service` por constructor (inyección). Su responsabilidad es **orquestrar un caso de uso completo**.

`app/services/crear_requerimiento_service.py`

```
@dataclass
class CrearRequerimientoService:
    requerimiento_repo: IRequerimientoRepository
    usuario_repo: IUsuarioRepository
    factory: RequerimientoFactory
    notificacion_service: Optional[object] = None
```

`crear_incidente(solicitante_id, servicio, titulo, descripcion, urgencia, categoria) :`

1. `_validar_solicitante(solicitante_id)` → busca en repo, verifica `isinstance(Solicitante)`.
2. `_tiene_servicio_activo(solicitante, servicio) :`
 - Chequea suscripción estática(`solicitante.esta_suscripto(servicio)`).
 - Si no tiene estática, calcula: `altas_resueltas > bajas_resueltas` (solicitudes de alta/baja resueltas para ese servicio).
3. `self.factory.crear_incidente(...)` → Factory genera el Incidente con estado ABIERTO.
4. `incidente.numero = self.requerimiento_repo.siguiente_numero()` → número secuencial.
5. `self.requerimiento_repo.guardar(incidente)` → persiste.
6. `self.notificacion_service.notificar_operadores(...)` → notifica a todos los operadores.
7. Retorna el incidente.

`crear_solicitud(solicitante_id, servicio, titulo, descripcion, categoria) :`

Similar pero con validaciones adicionales de alta/baja:

- **Alta:** ¿Ya tiene solicitud de alta **pendiente** (ABIERTO/ASIGNADO/REABIERTO) para ese servicio? → Error. ¿Ya tiene el servicio activo? → Error.
- **Baja:** ¿Tiene el servicio activo? Si no → Error.

`app/services/asignar_tecnico_service.py`

```

def asignar(self, requerimiento_id, operador_id, tecnico_id) -> Requerimiento:
    requerimiento = self.requerimiento_repo.obtener_por_id(requerimiento_id)
    operador = self.usuario_repo.obtener_por_id(operador_id)          # verificar
    tecnico = self.usuario_repo.obtener_por_id(tecnico_id)            # verificar
    operador.asignar(requerimiento, tecnico)                            # actualizar
    self.requerimiento_repo.guardar(requerimiento)
    # Notifica supervisores + técnico asignado
    return requerimiento

def asignar_con_estrategia(self, requerimiento_id, operador_id,
                           estrategia: EstrategiaAsignacion, candidatos=[])
    # Strategy Pattern: tecnico = estrategia.elegir_tecnico(req, candidatos)
    # Luego asigna igual que arriba

```

Nota: `asignar_con_estrategia()` acepta un objeto `EstrategiaAsignacion` (por especialidad o por carga). Es la implementación del **Strategy Pattern** en la capa de servicio.

app/services/derivar_requerimiento_service.py

```

def derivar(self, requerimiento_id, tecnico_origen_id, tecnico_destino_id):
    # Busca req, técnico origen, técnico destino
    # Determina actor real (puede ser un Operador/Supervisor que ordena la derivación)
    tecnico_origen.derivar(requerimiento, tecnico_destino, motivo, actor_real)
    self.requerimiento_repo.guardar(requerimiento)
    # Notifica supervisores + técnico destino

```

app/services/comentario_service.py

```

def agregar_comentario(self, requerimiento_id, autor_id, cuerpo):
    requerimiento = self.requerimiento_repo.obtener_por_id(requerimiento_id)
    usuario = self.usuario_repo.obtener_por_id(autor_id)
    if not usuario.puede_comentar(requerimiento):                    # POLIMORFISMO
        raise PermissionDeniedError(f"El usuario {usuario.nombre} no tiene permiso")
    requerimiento.agregar_comentario(autor_id=autor_id, autor_nombre=usuario.nombre, cuerpo=cuerpo)
    self.requerimiento_repo.guardar(requerimiento)
    # Notifica supervisores; si autor es Solicitante y hay técnico asignado

```

app/services/resolver_requerimiento_service.py

```

def resolver(self, requerimiento_id, tecnico_id):
    requerimiento = self.requerimiento_repo.obtener_por_id(requerimiento_id)
    tecnico = self.usuario_repo.obtener_por_id(tecnico_id)
    tecnico.resolver(requerimiento)    # State Pattern: ASIGNADO → RESUELI
    self.requerimiento_repo.guardar(requerimiento)
    # Notifica supervisores

```

app/services/reabrir_requerimiento_service.py

```

def reabrir(self, requerimiento_id, actor_id, motivo):
    requerimiento = self.requerimiento_repo.obtener_por_id(requerimiento_id)
    actor = self.usuario_repo.obtener_por_id(actor_id)    # verifica isins
    requerimiento.reabrir(actor_id=actor_id, actor_nombre=actor.nombre, m
    self.requerimiento_repo.guardar(requerimiento)        # State Pattern
    # Notifica supervisores

```

app/services/notificacion_service.py

```

@dataclass
class NotificacionService:
    usuario_repo: IUsuarioRepository
    notificacion_repo: INotificacionRepository

    def notificar_accion(self, actor_id, actor_nombre, requerimiento_id,
        """Busca supervisores que monitorean al actor y crea notificación
        supervisores = self.usuario_repo.listar_supervisores_de(actor_id)
        for sup in supervisores:
            n = self._crear_notificacion(destinatario_id=sup.id, ...)
            sup.recibir_notificacion(n)

    def notificar_operadores(self, actor_id, actor_nombre, requerimiento_id,
        """Notifica a todos los operadores (ej: nuevo requerimiento)."""

    def notificar_usuario(self, destinatario_id, actor_id, actor_nombre,
        """Notificación directa a un usuario (ej: al técnico cuando le as

    def listar_notificaciones(self, usuario_id): ...
    def listar_no_leidas(self, usuario_id): ...
    def marcar_como_leida(self, notificacion_id): ...

```

1.10 Repositorios (`app/repositories/`)

`app/repositories/base.py` – Interfaces abstractas

```
class IRequerimientoRepository(ABC):
    def guardar(self, requerimiento) -> None: ...
    def obtener_por_id(self, id: UUID): ...
    def listar_todos(self) -> List: ...
    def listar_por_autor(self, autor_id: UUID) -> List: ...
    def listar_por_tecnico_asignado(self, tecnico_id: UUID) -> List: ...
    def siguiente_numero(self) -> int:
        return len(self.listar_todos()) + 1

class IUsuarioRepository(ABC):
    def guardar(self, usuario) -> None: ...
    def obtener_por_id(self, id: UUID): ...
    def obtener_por_email(self, email: str): ...
    def listar_tecnicos(self) -> List: ...
    def listar_supervisores_de(self, usuario_id: UUID) -> List: ...
    def listar_todos(self) -> List: ...

class INotificacionRepository(ABC):
    def guardar(self, notificacion) -> None: ...
    def listar_por_destinatario(self, destinatario_id: UUID) -> List: ...
    def listar_no_leidas(self, destinatario_id: UUID) -> List: ...
    def obtener_por_id(self, id: UUID): ...
```

`app/repositories/in_memory/` – Para tests

Implementaciones con `Dict[UUID, T]` en memoria. Ejemplo:

```
class RequerimientoRepositoryInMemory(IRequerimientoRepository):
    def __init__(self):
        self._storage: Dict[UUID, Requerimiento] = {}
    def guardar(self, requerimiento): self._storage[requerimiento.id] = r
    def obtener_por_id(self, id): return self._storage.get(id)
    def listar_todos(self): return list(self._storage.values())
```

`app/repositories/mongo/` – Producción

- `UsuarioRepositoryMongo` : Usa `UsuarioDbSchema.to_mongo()` y `.from_mongo()` para convertir.
- `RequerimientoRepositoryMongo` : Usa `RequerimientoDbSchema.to_mongo()` y `.from_mongo()`.
- `NotificacionRepositoryMongo` : Usa `NotificacionDbSchema.to_mongo()` y `.from_mongo()`.

app/repositories/mongo/schemas_mongo.py – Conversores dominio ↔ MongoDB

`UsuarioDbSchema.to_mongo(usuario)` : Serializa `Usuario` a dict BSON con `_id`, `tipo` discriminador, campos base y campos específicos (suscripciones para Solicitante, especialidades para Técnico, monitoreados para Supervisor).

`UsuarioDbSchema.from_mongo(doc)` : Según `doc["tipo"]` instancia `Solicitante`, `Operador`, `Tecnico` o `Supervisor`. Reconstruye suscripciones, especialidades y monitoreados manualmente.

`RequerimientoDbSchema.to_mongo(req)` : Serializa con mapeos estáticos para Value Objects:

```
_URG_TO_STR = {UrgenciaCritica: "critica", UrgenciaImportante: "important"}
_CAT_INC_TO_STR = {ServicioInaccesible: "servicio_inaccesible", BloqueoSI: "bloqueo_si"}
_STR_TO_ESTADO = {"ABIERTO": EstadoAbierto, "ASIGNADO": EstadoAsignado, "CERRADO": EstadoCerrado}
```

`RequerimientoDbSchema.from_mongo(doc)` : Reconstruye la entidad completa. **Importante:** limpia los datos generados por `__post_init__` y restaura desde el documento:

```
req._eventos.clear()
req._comentarios.clear()
req._historial_asignaciones.clear()
# Luego reconstruye cada lista desde el documento MongoDB
```

1.11 Schemas Pydantic (app/schemas/)

app/schemas/requerimiento_schemas.py

Requests:

Schema	Campos
CrearIncidenteRequest	servicio_nombre (pattern regex), titulo (min 1, max 200), descripcion (min 1), urgencia (pattern: critical importante menor), categoria (pattern)
CrearSolicitudRequest	servicio_nombre, titulo, descripcion, categoria (alta_servicio baja_servicio)
AsignarTecnicoRequest	tecnico_id: UUID
DerivarRequerimientoRequest	tecnico_destino_id: UUID, motivo: str (min 1)
AgregarComentarioRequest	cuerpo: str (min 1)
ReabrirRequerimientoRequest	motivo: str (min 1)
ResolverRequerimientoRequest	(vacío — no necesita body)

Responses:

Schema	Campos principales
RequerimientoResponse	id, numero, tipo, servicio, titulo, descripcion, autor_id, autor_nombre, creado_en, estado, resuelto_en, urgencia, categoria, asignacion_actual, comentarios[], eventos[]
ComentarioResponse	autor_id, autor_nombre, cuerpo, fecha
EventoResponse	tipo, actor_id, actor_nombre, fecha, detalle
AsignacionResponse	tecnico_id, tecnico_nombre, asignado_por_id, asignado_por_nombre, desde, hasta

app/schemas/usuario_schemas.py

Schema Request	Campos
CrearSolicitanteRequest	nombre (1-100), email: EmailStr, password (min 6)

Schema Request	Campos
CrearOperadorRequest	nombre, email: EmailStr, password
CrearTecnicoRequest	nombre, email: EmailStr, password, especialidades_ids: List[UUID]
CrearSupervisorRequest	nombre, email: EmailStr, password
AgregarSuscripcionRequest	servicio_nombre (pattern regex)
MonitorearEmpleadoRequest	empleado_id: UUID

Schema Response	Hereda de	Extra
UsuarioResponse	—	id, nombre, email, tipo, creado_en
SolicitanteResponse	UsuarioResponse	suscripciones[]
TecnicoResponse	UsuarioResponse	especialidades[]
SupervisorResponse	UsuarioResponse	monitoreados_ids[]

app/schemas/auth_schemas.py

Schema	Campos
LoginRequest	email: EmailStr, password: str (min 1)
TokenResponse	access token: str, token_type: str = "bearer"
UsuarioAutenticadoResponse	id, nombre, email, rol

1.12 Infraestructura (app/infrastructure/)

app/infrastructure/database.py – Singleton

```
class DatabaseConnection:
    _instance: Optional["DatabaseConnection"] = None
    _client: Optional[MongoClient] = None
```

```

_db: Optional[Database] = None

def __new__(cls) -> "DatabaseConnection":
    if cls._instance is None:
        cls._instance = super().__new__(cls)
    return cls._instance

def connect(self): ...
def get_database(self) -> Database: ...
def get_collection(self, name: str) -> Collection: ...

_db_connection = DatabaseConnection() # Singleton

def get_database() -> Database: return _db_connection.get_database()
def get_collection(name: str) -> Collection: return _db_connection.get_cc

```

¿Por qué Singleton? Necesito exactamente una conexión a MongoDB compartida. Crear múltiples conexiones desperdicia recursos. `DatabaseConnection` garantiza que `__new__` siempre retorna la misma instancia.

app/config/settings.py

```

class Settings:
    MONGO_URI: str = os.getenv("MONGO_URI", "mongodb://root:example@local
    MONGO_DB: str = os.getenv("MONGO_DB", "mesa_ayuda")
    USE_MONGO: bool = os.getenv("USE_MONGO", "true").lower() == "true"
    JWT_SECRET_KEY: str = os.getenv("JWT_SECRET_KEY", "super-secret-ke
    JWT_ALGORITHM: str = os.getenv("JWT_ALGORITHM", "HS256")
    JWT_EXPIRE_MINUTES: int = int(os.getenv("JWT_EXPIRE_MINUTES", "60"))

settings = Settings()

```

PARTE 2 – FLUJO COMPLETO DE CADA ENDPOINT

2.1 GET /health – Health Check

Archivo: `app/routers/health_router.py`

Función: `health_check()`

Auth: No requiere

Response: `{"status": "ok", "message": "Mesa de Ayuda API funcionando"}`

Paso 1: GET /health → FastAPI

Paso 2: No hay body → no hay validación

Paso 3: No hay dependencias de seguridad

Paso 4: `health_check()` retorna dict literal

Paso 5: FastAPI serializa a JSON con status 200

2.2 POST /auth/login – Login JWT

Archivo: `app/routers/auth_router.py`

Función: `login(request: LoginRequest)`

Schema request: `LoginRequest` → `{email: EmailStr, password: str}`

Dependencia: Ninguna (público)

Schema response: `TokenResponse` → `{access_token: str, token_type: "bearer"}`

Paso 1: FastAPI recibe POST /auth/login con body JSON
`{"email": "maria@test.com", "password": "password123"}`

Paso 2: Pydantic valida con `LoginRequest`:
- email debe ser `EmailStr` válido
- password debe ser string no vacío (`min_length=1`)
Si falla → 422 Unprocessable Entity automáticamente

Paso 3: No hay dependencias de seguridad (endpoint público)

Paso 4: `login(request)` ejecuta:

- a. `repo = get_usuario_repo()`
- b. `usuario = repo.obtener_por_email(request.email)`
- c. Si `usuario is None` → `HTTPException(401, "Credenciales invalidas")`
- d. `verify_password(request.password, usuario.password_hash)`
→ Si `False` → `HTTPException(401, "Credenciales invalidas")`
- e. `usuario.actualizar_ultimo_acceso()`
- f. `repo.guardar(usuario)` – actualiza último acceso
- g. `rol = usuario.__class__.__name__` – "Solicitante", "Operador", etc.

```
h. token = create_access_token(user_id=usuario.id,
email=usuario.email, rol=rol)
```

Paso 5: Retorna TokenResponse(access_token=token)

Paso 6: FastAPI serializa a JSON con status 200

2.3 GET /auth/me – Usuario actual

Archivo: `app/routers/auth_router.py`

Función: `me(usuario=Depends(get_current_user))`

Dependencia: `get_current_user`

Schema response: `UsuarioAutenticadoResponse`

Paso 1: GET /auth/me con header Authorization: Bearer eyJhbG...

Paso 2: No hay body

Paso 3: DEPENDENCIA `get_current_user`:

- a. HTTPBearer extrae token del header
- b. `decode_access_token(token)` → verifica firma + expiración
→ Si `JWTErrror` → `HTTPException(401)`
- c. `user_id = UUID(payload["sub"])`
- d. `repo.obtener_por_id(user_id)`
→ Si `None` → `HTTPException(401)`
- e. Retorna entidad de dominio (Solicitante, Operador, etc.)

Paso 4: `me()` retorna `UsuarioAutenticadoResponse(`
 `id=usuario.id, nombre=usuario.nombre,`
 `email=usuario.email, rol=usuario.__class__.__name__`
 `)`

Paso 5: FastAPI serializa con status 200

2.4 POST /usuarios/solicitantes – Crear solicitante (público)

Archivo: `app/routers/usuarios_router.py`

Función: `crear_solicitante(request: CrearSolicitanteRequest)`

Schema request: `CrearSolicitanteRequest` → {nombre, email, password}

Dependencia: Ninguna (registro público)

Entidad creada: `Solicitante`

Schema response: `SolicitanteResponse` (status 201)

Paso 1: `POST /usuarios/solicitantes`

Paso 2: Pydantic valida:

- nombre: min_length=1, max_length=100
 - email: EmailStr
 - password: min_length=6
- Si falla → 422

Paso 3: No hay auth (público)

Paso 4: `crear_solicitante():`

- a. `repo = get_usuario_repo()`
- b. `_check_email_libre(repo, request.email)`
→ `repo.obtener_por_email()` → si existe → `HTTPException(400)`
- c. `solicitante = Solicitante(`
 `id=uuid4(), nombre=request.nombre, email=request.email,`
 `password_hash=hash_password(request.password),`
 `creado_en=_now(), ultimo_acceso=_now(),`
 `)`
→ `__post_init__` valida email y nombre
- d. `repo.guardar(solicitante)`

Paso 5: `_usuario_to_response(solicitante)` → `SolicitanteResponse`

Paso 6: FastAPI serializa con status 201

2.5 POST `/usuarios/operadores` – Crear operador (protegido)

Archivo: `app/routers/usuarios_router.py`

Función: `crear_operador(request,`
`_usuario=Depends(require_roles(["Supervisor"])))`

Dependencia: `require_roles(["Supervisor"])`

Entidad creada: `Operador`

Paso 1: `POST /usuarios/operadores` con JWT de un Supervisor

Paso 2: `Pydantic` valida `CrearOperadorRequest`

Paso 3: `require_roles(["Supervisor"])` verifica JWT y rol

Paso 4: `crear_operador()`:

- a. `_check_email_libre()`
- b. `Operador(nombre, email, ...)` → `__post_init__` valida:
 - email con `@comunicarlos.com.ar` (dominio)
 - Si no → `ValueError` → `HTTPException(400)`
- c. `repo.guardar(operador)`
- d. `_auto_monitorear_empleado(repo, operador.id)`
 - Busca TODOS los supervisores existentes
 - Agrega al operador a su lista de monitoreados
 - Persiste cada supervisor modificado

Paso 5: `_usuario_to_response(operador)` → `UsuarioResponse`

Paso 6: `Status 201`

2.6 POST `/requerimientos/incidentes` – Crear incidente

Archivo: `app/routers/requerimientos_router.py`

Función: `crear_incidente(request: CrearIncidenteRequest, usuario=Depends(require_roles(["Solicitante"])))`

Schema request: `CrearIncidenteRequest`

Dependencia: `require_roles(["Solicitante"])`

Service: `CrearRequerimientoService` vía `get_crear_requerimiento_service()`

Factory: `RequerimientoFactory.crear_incidente()`

Entidad creada: `Incidente(Requerimiento)` con `EstadoAbierto`

Repositorio: `IRequerimientoRepository.guardar()`

Schema response: `RequerimientoResponse` (status 201)

Flujo paso a paso detallado:

Paso 1: FastAPI recibe POST /requerimientos/incidentes

```
Body: {
    "servicio_nombre": "internet",
    "titulo": "Sin conexión",
    "descripcion": "No tengo internet desde las 10am",
    "urgencia": "critica",
    "categoria": "servicio_inaccesible"
}

Header: Authorization: Bearer eyJhbG...
```

Paso 2: Pydantic valida CrearIncidenteRequest:

```
- servicio_nombre: pattern regex "^(internet|telefonía|tv)$"
- titulo: min_length=1, max_length=200
- descripcion: min_length=1
- urgencia: pattern "^(critica|importante|menor)$"
- categoria: pattern
"^(servicio_inaccesible|bloqueo_sim|perdida_equipo)$"
Si falla → 422 automático
```

Paso 3: DEPENDENCIA require_rol(["Solicitante"]):

- a. HTTPBearer extrae token
- b. decode_access_token verifica JWT
- c. Busca usuario en repositorio
- d. Verifica clase.__name__ == "Solicitante" → si es Operador →

403

Paso 4: ROUTER crear_incidente() comienza:

- a. service = get_crear_requerimiento_service()
→ dependencies.py instancia CrearRequerimientoService con repos inyectados

- b. servicio = obtener_servicio(request.servicio_nombre)
→ Value Object Servicio validado contra catálogo
→ Si nombre inválido → ValueError → HTTPException(400)

- c. urgencia = _map_urgencia(request.urgencia)
→ "critica" → UrgenciaCritica()
→ "importante" → UrgenciaImportante()
→ "menor" → UrgenciaMenor()

```

d. categoria = _map_categoria_incidente(request.categoria)
   → "servicio_inaccesible" → ServicioInaccesible()
   → "bloqueo_sim" → BloqueoSIM()
   → "perdida_equipo" → PerdidaEquipo()

e. VALIDACIÓN COMPATIBILIDAD EN ROUTER:
   categoria.es_compatible_con(servicio.nombre)
   → BloqueoSIM solo es compatible con "telefonía"
   → Si internet + bloqueo_sim → HTTPException(400, "no compatible")

```

Paso 5: ROUTER delega a SERVICE:

```

service.crear_incidente(
    solicitante_id=usuario.id, servicio=servicio,
    titulo=request.titulo, descripcion=request.descripcion,
    urgencia=urgencia, categoria=categoria,
)

```

Paso 6: SERVICE ejecuta:

```

a. solicitante = _validar_solicitante(solicitante_id)
   → repo.obtener_por_id() → isinstance(Solicitante)

b. _tiene_servicio_activo(solicitante, servicio):
   → Chequea suscripción estática 0 (altas_resueltas >
bajas_resueltas)
   → Si no activo → DomainRuleError("no esta suscripto")

```

Paso 7: SERVICE usa FACTORY:

```

incidente = self.factory.crear_incidente(
    autor=solicitante, servicio=servicio, titulo=titulo,
    descripcion=descripcion, urgencia=urgencia,
categoria=categoria,
)

```

FACTORY internamente:

```

- Genera UUID con uuid4()
- estado = EstadoAbierto()
- Incidente.__post_init__() →
    valida urgencia no None
    valida categoria no None
    Requerimiento.__post_init__() →

```

```
        _validar_titulo()
        _validar_descripcion()
        _registrar_evento(tipo="CREACION", ...)
```

Paso 8: SERVICE asigna número secuencial:

```
incidente.numero = self.requerimiento_repo.siguiente_numero()
```

Paso 9: SERVICE persiste:

```
self.requerimiento_repo.guardar(incidente)
```

→ Si MongoDB: RequerimientoDbSchema.to_mongo() convierte entidad a doc BSON

```
con replace_one(..., upsert=True)
```

Paso 10: SERVICE notifica operadores:

```
self.notificacion_service.notificar_operadores(
    actor_id=solicitante.id, actor_nombre=solicitante.nombre,
    requerimiento_id=incidente.id,
    mensaje=f"Nuevo incidente #{incidente.numero}: {titulo}"
)
```

→ Busca todos los Operadores en usuario_repo

→ Crea Notificacion para cada uno

→ Persiste en notificacion_repo

Paso 11: SERVICE retorna incidente al ROUTER

Paso 12: ROUTER convierte a response:

```
_requerimiento_to_response(incidente) → RequerimientoResponse
- tipo: "incidente"
- estado: req.estado.nombre() → "ABIERTO"
- urgencia: req.urgencia.nombre() → "critica"
- categoria: req.categoria.nombre() → "servicio_inaccesible"
- asignacion_actual: None (recién creado)
- comentarios: []
- eventos: [{"tipo": "CREACION", ...}]
```

Paso 13: FastAPI serializa RequerimientoResponse a JSON

Retorna con status 201 Created

2.7 POST /requerimientos/solicitudes – Crear solicitud

Archivo: `app/routers/requerimientos_router.py`

Función: `crear_solicitud(request: CrearSolicitudRequest, usuario=Depends(require_roles(["Solicitante"])))`

Mismo flujo que crear incidente EXCEPTO:

- No hay campo urgencia ni en el request ni en la entidad
 - `_map_categoria_solicitud() : "alta_servicio" → AltaServicio() , "baja_servicio" → BajaServicio()`
 - El service valida lógica adicional: no duplicar altas pendientes, no dar de baja servicio inactivo
 - Factory crea `Solicitud` en lugar de `Incidente`
-

2.8 GET /requerimientos – Listar requerimientos (filtrado por rol)

Archivo: `app/routers/requerimientos_router.py`

Función: `listar_requerimientos(_usuario=Depends(get_current_user))`

Dependencia: `get_current_user` – cualquier usuario autenticado

Paso 1: `GET /requerimientos` con JWT válido

Paso 2: `get_current_user` verifica token y retorna entidad

Paso 3: `repo.listar_todos()`

Paso 4: FILTRADO POR ROL:

```
if isinstance(_usuario, Solicitante):
    requerimientos = [r for r in requerimientos if r.autor_id ==
_usuario.id]
elif isinstance(_usuario, Tecnico):
    requerimientos = [r for r in requerimientos if
r.asignado_a_id() == _usuario.id]
# Operador/Supervisor: ven TODOS sin filtrar
```

Paso 5: `[_requerimiento_to_response(r) for r in requerimientos]`
Retorna lista JSON con status 200

2.9 GET `/requerimientos/{id}` – Detalle

Archivo: `app/routers/requerimientos_router.py`

Función: `obtener_requerimiento(requerimiento_id: UUID, _usuario=Depends(get_current_user))`

Paso 1: `GET /requerimientos/{uuid}`
Paso 2: FastAPI convierte path param a UUID
Paso 3: `get_current_user` verifica JWT
Paso 4: `repo.obtener_por_id(requerimiento_id) → si None → 404`
Paso 5: Solicitante solo ve propios: `if isinstance(Solicitante) and req.autor_id != _usuario.id → 403`
Paso 6: `_requerimiento_to_response()` → status 200

2.10 POST `/requerimientos/{id}/asignar` – Asignar técnico

Archivo: `app/routers/requerimientos_router.py`

Función: `asignar_tecnico(requerimiento_id, request: AsignarTecnicoRequest, usuario=Depends(require_roles(["Operador"])))`

Service: `AsignarTecnicoService`

State Pattern: `EstadoAbierto/EstadoReabierto → EstadoAsignado`

Paso 1: `POST /requerimientos/{uuid}/asignar`
Body: `{"tecnico_id": "uuid-del-tecnico"}`
Header: Bearer token de un Operador

Paso 2: Pydantic valida `AsignarTecnicoRequest` (`tecnico_id: UUID`)

Paso 3: `require_roles(["Operador"])` verifica rol

Paso 4: `service = get_asignar_tecnico_service()`
`requerimiento = service.asignar(requerimiento_id, operador_id=usuario.id, tecnico_id=request.tecnico_id)`

)

Paso 5: SERVICE ejecuta:

```
a. req = requerimiento_repo.obtener_por_id(requerimiento_id)
b. operador = usuario_repo.obtener_por_id(operador_id) →
isinstance(Operador)
c. tecnico = usuario_repo.obtener_por_id(tecnico_id) →
isinstance(Tecnico)
```

Paso 6: SERVICE → DOMINIO:

```
operador.asignar(requerimiento, tecnico)
→ req.asignar(tecnico_id, tecnico_nombre, asignado_por_id,
asignado_por_nombre)
```

Paso 7: STATE PATTERN – req.estado (EstadoAbierto):

```
a. self.estado.permite_asignar() → True 
b. _cerrar_asignacion_actual() → cierra previa si existe
c. Nueva Asignacion(tecnico_id, tecnico_nombre, asignado_por_id,
...)
d. req.asignacion_actual = nueva
e. req._historial_asignaciones.append(nueva)
f. req.estado = EstadoAsignado() ← CAMBIO DE ESTADO
g. _registrar_evento(tipo="ASIGNACION", detalle=f"Asignado a
{tecnico_nombre}")
```

SI req.estado fuera EstadoResuelto:

```
→ permite_asignar() → False
→ DomainRuleError("El estado RESUELTO no permite asignar")
```

Paso 8: requerimiento_repo.guardar(req)

Paso 9: Notifica supervisores + técnico asignado

Paso 10: _requerimiento_to_response(req) con estado "ASIGNADO"

FastAPI retorna JSON status 200

2.11 POST /requerimientos/{id}/derivar – Derivar

Archivo: `app/routers/requerimientos_router.py`

Función: `derivar_requerimiento(requerimiento_id, request: DerivarRequerimientoRequest, usuario=Depends(require_roles(["Tecnico"])))`

Service: `DerivarRequerimientoService`

Paso 1: `POST /requerimientos/{uuid}/derivar`

Body: `{"tecnico_destino_id": "uuid", "motivo": "No es mi especialidad"}`

Paso 2: `Pydantic valida motivo min_length=1`

Paso 3: `require_roles(["Tecnico"])`

Paso 4: `service.derivar()` ejecuta:

a. Busca req, técnico origen (usuario autenticado), técnico destino

b. `tecnico_origen.derivar(req, tecnico_destino, motivo, ...)`
→ `req.derivar(origen_id, origen_nombre, destino_id, destino_nombre, motivo)`

→ Valida: `asignacion_actual.tecnico_id == origen_id`

→ Valida: `origen_id != destino_id`

→ `_cerrar_asignacion_actual()` → pone fecha hasta

→ Nueva Asignacion para técnico destino

→ Evento DERIVACION

→ Estado NO CAMBIA (sigue ASIGNADO)

Paso 5: `Persiste y notifica`

Paso 6: `Response con nueva asignación, mismo estado "ASIGNADO"`

2.12 POST `/requerimientos/{id}/comentarios`

– Comentar

Archivo: `app/routers/requerimientos_router.py`

Función: `agregar_comentario(requerimiento_id, request: AgregarComentarioRequest, usuario=Depends(require_roles(["Solicitante", "Operador", "Tecnico"])))`

Service: `ComentarioService`

Paso 1: `POST /requerimientos/{uuid}/comentarios`
Body: `{"cuerpo": "Estamos trabajando en el problema"}`

Paso 2: Pydantic valida cuerpo `min_length=1`

Paso 3: `require_roles EXCLUYE Supervisor`
→ Si un Supervisor intenta → 403

Paso 4: `service.agregar_comentario():`
a. Busca req y usuario
b. POLIMORFISMO: `usuario.puede_comentar(requerimiento)`
- Solicitante: `req.autor_id == usuario.id`
- Operador: `True` (siempre)
- Tecnico: `req.asignado_a_id() == usuario.id`
→ Si False → `PermissionDeniedError`

Paso 5: `req.agregar_comentario(autor_id, autor_nombre, cuerpo)`
→ Crea `Comentario(frozen=True)` inmutable
→ Agrega a `_comentarios`
→ `_registrar_evento(tipo="COMENTARIO", ...)`

Paso 6: Persiste

Paso 7: Notifica supervisores. Si autor es Solicitante y hay técnico → notifica técnico

Paso 8: Response con nuevo comentario en la lista

Manejo de errores:

```
except PermissionDeniedError as e:
    raise HTTPException(status_code=403, detail=str(e))
except DomainRuleError as e:
    raise HTTPException(status_code=400, detail=str(e))
```

2.13 POST /requerimientos/{id}/resolver – Resolver

Archivo: `app/routers/requerimientos_router.py`

Función: `resolver_requerimiento(requerimiento_id, usuario=Depends(require_roles(["Tecnico"])))`

Service: `ResolverRequerimientoService`

State Pattern: `EstadoAsignado/EstadoReabierto → EstadoResuelto`

Paso 1: `POST /requerimientos/{uuid}/resolver` (sin body)

Paso 2: `require_roles(["Tecnico"])`

Paso 3: `service.resolver(requerimiento_id, tecnico_id=usuario.id)`

Paso 4: **SERVICE:**

a. Busca req y técnico

b. `tecnico.resolver(req)`

→ Valida `puede_ver(req): req.asignado_a_id() == self.id`

→ `req.resolver(tecnico_id, tecnico_nombre)`

Paso 5: **STATE PATTERN** – req.estado es EstadoAsignado:

a. `permite_resolver()` → True 

b. Valida `asignacion_actual.tecnico_id == tecnico_id`

c. `req.resuelto_en = datetime.now(timezone.utc)`

d. `req.estado = EstadoResuelto()` ← CAMBIO DE ESTADO

e. Evento RESOLUCION

SI el estado fuera EstadoAbierto:

→ `permite_resolver()` → False

→ `DomainRuleError("El estado ABIERTO no permite resolver")`

Paso 6: Persiste → Notifica

Paso 7: Response: estado "RESUELTO", resuelto_en con fecha

2.14 POST `/requerimientos/{id}/reabrir` – Reabrir

Archivo: `app/routers/requerimientos_router.py`

Función: `reabrir_requerimiento(requerimiento_id, request:`

`ReabrirRequerimientoRequest, usuario=Depends(require_roles(["Operador",`

```
"Tecnico"])))
```

Service: `ReabrirRequerimientoService`

State Pattern: `EstadoResuelto` → `EstadoReabierto`

Paso 1: `POST /requerimientos/{uuid}/reabrir`

Body: `{"motivo": "El problema persiste"}`

Paso 2: `require_roles(["Operador", "Tecnico"])`

→ Solicitantes NO pueden reabrir

Paso 3: `service.reabrir(requerimiento_id, actor_id=usuario.id, motivo)`

Paso 4: SERVICE:

a. Busca req y actor → `isinstance(Operador, Tecnico)`

b. `req.reabrir(actor_id, actor_nombre, motivo)`

Paso 5: STATE PATTERN – req.estado debe ser `EstadoResuelto`:

a. `permite_reabrir()` → True 

b. `req.resuelto_en = None`

c. `req.estado = EstadoReabierto()` ← CAMBIO DE ESTADO

d. Evento REAPERTURA con motivo

Paso 6: Persiste → Notifica

Paso 7: Response: estado "REABIERTO", resuelto_en null

2.15 Notificaciones

GET /notificaciones – Listar

Archivo: `app/routers/notificaciones_router.py`

Función: `listar_notificaciones(_usuario=Depends(require_roles(["Supervisor", "Operador", "Tecnico"])))`

→ `service.listar_notificaciones(_usuario.id)`

→ Ordena por fecha descendente

→ Lista `[NotificacionResponse]`

GET /notificaciones/no-leidas – Solo no leídas

→ `service.listar_no_leidas(_usuario.id)`

GET /notificaciones/contador – Cantidad

→ `ContadorResponse(no_leidas=len(service.listar_no_leidas(...)))`

PATCH /notificaciones/{id}/leer – Marcar como leída

Paso 1: `PATCH /notificaciones/{uuid}/leer`

Paso 2: `service.marcar_como_leida(notificacion_id)`

→ `notificacion.marcar_leida()` → `self.leida = True`

→ Persiste

Paso 3: Valida: `notificacion.destinatario_id == _usuario.id`

→ Si no → 403 (no puede marcar notificación de otro)

Paso 4: `NotificacionResponse` con `leida=True`

PATCH /notificaciones/leer-todas – Marcar todas

→ `service.listar_no_leidas(_usuario.id)`

→ `for notif in no_leidas: service.marcar_como_leida(notif.id)`

→ `{"mensaje": "N notificaciones marcadas como leidas"}`

2.16 GET /servicios – Catálogo público

Archivo: `app/routers/servicios_router.py`

Función: `obtener_catalogo_servicios()`

Auth: No requiere

→ `listar_servicios()` → 3 Value Objects Servicio

→ `[ServicioCatalogoResponse(id, nombre) for s in servicios]`

2.17 Otros endpoints de usuarios

POST /usuarios/tecnicos – Crear técnico

Dependencia: `require_roles(["Supervisor"])`

Entidad: `Tecnico` → valida email @comunicarlos.com.ar

Auto-monitoreo: agrega a todos los supervisores

Status: 201

POST /usuarios/supervisores – Crear supervisor

Dependencia: `require_roles(["Supervisor"])`

Entidad: Supervisor → valida email @comunicarlos.com.ar

Status: 201

GET /usuarios – Listar todos

Dependencia: `require_roles(["Supervisor"])`

→ `[_usuario_to_response(u) for u in repo.listar_todos()]`

GET /usuarios/tecnicos – Listar técnicos

Dependencia: `require_roles(["Operador", "Supervisor", "Tecnico"])`

→ `repo.listar_tecnicos()` → solo Técnicos

GET /usuarios/{id} – Obtener por ID

Dependencia: `get_current_user`

Solicitantes y Técnicos solo pueden verse a sí mismos → 403

POST /usuarios/me/suscripciones – Suscribirse a servicio

Dependencia: `require_roles(["Solicitante"])`

→ `servicio = obtener_servicio(request.servicio_nombre)`

→ `usuario.agregar_suscripcion(servicio)` → valida no duplicado (dominio)

→ `repo.guardar(usuario)`

→ `SolicitanteResponse` con suscripciones actualizadas

POST /usuarios/me/monitorear – Monitorear empleado

Dependencia: `require_roles(["Supervisor"])`

→ Valida: no puede monitorearse a sí mismo

→ Valida: empleado existe y es Operador o Tecnico

→ `supervisor.monitorear(empleado_id)` → agrega a lista (dominio)

→ `repo.guardar(supervisor)`

PARTE 3 – ANÁLISIS DE PATRONES Y PRINCIPIOS

3.1 State Pattern – Ciclo de vida del requerimiento

Implementación: Cada estado es una clase `@dataclass(frozen=True, slots=True)` que define qué transiciones permite. El `Requerimiento` delega a su estado actual preguntando `self.estado.permite_asignar()`, `self.estado.permite_resolver()`, `self.estado.permite_reabrir()`. Si la transición no está permitida → `DomainRuleError`.

Beneficio principal: Si se agrega un nuevo estado (ej: `EstadoEnEspera`), se crea UNA clase nueva sin modificar las existentes. Los tests existentes siguen pasando. **Open/Closed Principle.**

3.2 Uso de `@dataclass(frozen=True, slots=True)`

frozen=True: Hace la instancia inmutable. `obj.attr = value` lanza `FrozenInstanceError`. Se usa en:

- `Comentario` (registro histórico inmutable)
- `Evento` (hecho pasado inmutable)
- `Suscripcion` (snapshot estático)
- `Servicio`, `Urgencia`, `CategoriaIncidente`, `CategoriaSolicitud` (Value Objects)
- `EstadoRequerimiento` y sus subclasses

slots=True: Usa `__slots__` en vez de `__dict__`. Menor memoria por instancia (~30% menos), acceso más rápido, previene atributos dinámicos accidentales.

¿Por qué NO todas las entidades son frozen? Porque `Requerimiento` necesita mutar su estado, asignación y listas. `Notificacion` necesita `marcar_leida()`. Las entidades con ciclo de vida mutable no son frozen.

3.3 DomainRuleError y flujo de excepción

```
1. Dominio lanza: DomainRuleError("El estado ABIERTO no permite resolver")
2. Service NO la captura → propaga
3. Router captura:
    except DomainRuleError as e:
        raise HTTPException(status_code=400, detail=str(e))
4. FastAPI retorna: {"detail": "El estado ABIERTO no permite resolver"}
    Con status 400
```

¿Por qué el dominio no lanza HTTPException? Porque el dominio no sabe qué es HTTP. Si este dominio se usa en una CLI o un bot de Telegram, HTTPException no tendría sentido. DomainRuleError es agnóstica del protocolo.

3.4 Diferencia entre Schema y Entidad

Aspecto	Schema Pydantic	Entidad de Dominio
Propósito	Validar HTTP, serializar JSON	Modelar negocio, encapsular reglas
Lógica	Solo validación de formato	Reglas de negocio, transiciones de estado
Mutabilidad	Inmutable (DTO)	Mutable (ciclo de vida)
Dependencias	Pydantic	Python puro
Estado	str ("ABIERTO")	EstadoRequerimiento (objeto con métodos)
Ejemplo	RequerimientoResponse.estado: str	Requerimiento.estado: EstadoRequerimiento

3.5 ¿Por qué usar Services?

Sin services, el router haría TODO:

```
# MAL — router con toda la lógica
@router.post("/{id}/asignar")
def asignar(id, request, usuario):
    req = repo.obtener_por_id(id)
    tecnico = usuario_repo.obtener_por_id(request.tecnico_id)
    operador = usuario_repo.obtener_por_id(usuario.id)
    operador.asignar(req, tecnico)
    repo.guardar(req)
```

```
# crear notificaciones...
return _to_response(req)
```

Problemas: SRP violado (HTTP + negocio), no testeable sin FastAPI, no reutilizable.

Con services:

```
# BIEN — router delega
@router.post("/{id}/asignar")
def asignar(id, request, usuario):
    service = get_asignar_tecnico_service()
    req = service.asignar(id, usuario.id, request.tecnico_id)
    return _to_response(req)
```

El service se testea unitariamente con repositorios in-memory, sin HTTP. Los 29 tests de servicios prueban esto.

3.6 Responsabilidad del Repositorio

El repositorio es una **abstracción de persistencia**: guardar, recuperar, convertir entre dominio y almacenamiento.

Lo que el repositorio **NO** hace: validar reglas de negocio, ejecutar lógica de dominio, conocer HTTP.

IRequerimientoRepository (ABC)

```
├─ RequerimientoRepositoryInMemory → Dict Python (tests)
└─ RequerimientoRepositoryMongo    → Collection MongoDB (producción)
```

El cambio se hace en `dependencies.py` según `settings.USE_MONGO`.

3.7 Principios SOLID aplicados

S – Single Responsibility

Clase	Única responsabilidad
<code>EstadoAbierto</code>	Definir qué permite el estado "Abierto"
<code>CrearRequerimientoService</code>	Orquestar caso de uso "crear requerimiento"
<code>RequerimientoFactory</code>	Construir instancias de Incidente/Solicitud

Clase	Única responsabilidad
<code>UsuarioRepositoryMongo</code>	Persistir/recuperar usuarios de MongoDB
<code>RequerimientoResponse</code>	Serializar requerimiento para la API
<code>auth_dependencies.py</code>	Validar tokens JWT y roles

O – Open/Closed

- **Strategy**: Nueva estrategia de asignación → nueva clase, sin modificar existentes.
- **State**: Nuevo estado → nueva clase, sin buscar if/elif.
- **Factory**: Nuevo tipo de requerimiento → nuevo método, sin cambiar los existentes.

L – Liskov Substitution

- `Solicitante`, `Operador`, `Tecnico`, `Supervisor` son sustituibles donde se espera `Usuario`.
- `Incidente` y `Solicitud` donde se espera `Requerimiento`.
- `RequerimientoRepositoryMongo` donde se espera `IRequerimientoRepository`.

I – Interface Segregation

- `IUsuarioRepository` tiene métodos para usuarios.
- `IRequerimientoRepository` tiene métodos para requerimientos.
- `INotificacionRepository` tiene métodos para notificaciones.
- No hay mega-interfaz que obligue a implementar métodos innecesarios.

D – Dependency Inversion

- Los servicios dependen de **interfaces abstractas** (ABC), no de implementaciones.
- La inyección se resuelve en `dependencies.py`.
- El dominio **NO importa** FastAPI, MongoDB ni Pydantic.

3.8 Críticas al diseño (autocrítica honesta)

Aspecto	Crítica	Justificación
---------	---------	---------------

Aspecto	Crítica	Justificación
Creación de usuarios en router	No usa service dedicado – lógica directa en <code>usuarios_router.py</code>	La creación es simple. Un service sería over-engineering. Pero viola consistencia
Login en router	Lógica de auth en <code>auth_router.py</code> , no en un <code>AuthService</code>	Es infraestructura, no dominio
Compatibilidad categoría-servicio en router	<code>es_compatible_con()</code> se valida en el router antes de llamar al service	Debería ir en el service o la factory
Repos MongoDB con mucha lógica de conversión	<code>schemas_mongo.py</code> tiene métodos largos	Podría extraerse a un <code>Mapper</code> separado
No hay caching	Cada request busca usuario en BD	En producción debería cachearse
No hay transacciones	Si falla la notificación, el requerimiento ya se persistió	Debería usar transacciones MongoDB
EventBus sincrónico	<code>publicar()</code> ejecuta listeners sincrónicamente	En producción debería ser asíncrono
Filtrado en router	El listado de requerimientos filtra por rol en el router, no en el repo	Más eficiente sería filtrar en la query MongoDB

PARTE 4 – PREGUNTAS DE DEFENSA ORAL

4.1 – 20 Preguntas fuertes

1. ¿Por qué el router no modifica directamente el estado del requerimiento?

Porque el router es capa de presentación. Solo maneja HTTP: recibir requests, validar schemas, delegar a servicios y transformar responses. Si el router modificara el estado directamente, violaría SRP y haría imposible testear la lógica sin FastAPI. Los services encapsulan casos de uso y se testean con repos in-memory.

2. ¿Por qué el dominio no conoce FastAPI ni MongoDB?

Porque el dominio es la capa más estable. Si importara FastAPI, estaría acoplado a un framework web. Si cambio FastAPI por Django, solo cambio routers. Si cambio MongoDB por PostgreSQL, solo cambio repositorios. El dominio queda intacto. Los 82 tests de dominio pasan sin ningún cambio.

3. ¿Dónde se valida el input del usuario?

En dos capas:

- **Pydantic (schema):** formato HTTP — tipos, campos requeridos, min_length, regex patterns. Si falla → 422 automático.
- **Dominio:** reglas de negocio — ¿está suscripto?, ¿el estado permite?, ¿tiene permisos? Si falla → `DomainRuleError` → 400.

4. ¿Dónde se valida la regla de negocio?

En las entidades de dominio y en los servicios. Ejemplo: `Requerimiento.asignar()` consulta `self.estado.permite_asignar()`. Si es `EstadoResuelto` → `False` → `DomainRuleError("El estado RESUELTO no permite asignar")`. El router no sabe esta regla. Solo captura la excepción.

5. ¿Qué pasa si falla el dominio?

Lanza `DomainRuleError` o `PermissionDeniedError`. El service propaga. El router captura con try/except y convierte a `HTTPException(400)` o `HTTPException(403)`. FastAPI serializa como `{"detail": "mensaje"}`.

6. ¿Qué capa captura la excepción?

El router (presentación). Es quien traduce errores de dominio a errores HTTP. El service propaga. El dominio lanza.

7. ¿Por qué usás Factory en vez de crear el Incidente directamente?

Porque la creación requiere coordinar UUID, estado inicial, validación de campos y evento de creación. Si cada test y servicio creara incidentes manualmente, habría código duplicado y riesgo de olvidar un paso. La Factory centraliza y estandariza.

8. ¿Cómo garantizás que un Comentario no se modifica?

Con `@dataclass(frozen=True)`. Python impide reasignar atributos. `comentario.cuerpo = "otro"` lanza `FrozenInstanceError` en runtime. Es enforcement a nivel del lenguaje, no una convención.

9. ¿Qué ventaja tiene el Observer Pattern para notificaciones?

Desacoplamiento. El `EventBus` con `NotificacionListener` permite agregar comportamiento ante eventos sin modificar los servicios. Si mañana quiero un `EmailListener` o `SlackListener`, solo registro otro listener. Los servicios también usan `NotificacionService` directamente para el flujo principal.

10. ¿Cómo funciona la inyección de dependencias?

En `dependencies.py` (Composition Root) se crean las instancias concretas según `settings.USE_MONGO`. Los servicios reciben repos por constructor. Los routers los obtienen vía getter functions. Si cambio MongoDB por in-memory, solo cambio la variable de entorno o la condición en `dependencies.py`.

11. ¿Por qué `require_roles` es una función que retorna una función?

Es un closure. FastAPI necesita un callable para `Depends()`. `require_roles(["Operador"])` retorna una función que FastAPI ejecuta como dependencia. Internamente llama a `get_current_user` y verifica `usuario.__class__.__name__` contra `allowed_roles`.

12. ¿Cómo discriminás el tipo de usuario al leer de MongoDB?

El repositorio guarda un campo `"tipo"` en el documento (ej: `"Solicitante"`, `"Tecnico"`). En `UsuarioDbSchema.from_mongo()`, según `doc["tipo"]` se instancia la clase correcta. Esto permite reconstruir la jerarquía de herencia desde documentos planos.

13. ¿Por qué separar Incidente de Solicitud?

Tienen atributos diferentes: Incidente tiene `urgencia` y `CategoriaIncidente`, Solicitud tiene `CategoriaSolicitud` sin urgencia. Las reglas de creación difieren. La herencia comparte ciclo de vida (estados, asignaciones, comentarios).

14. ¿Por qué usar UUIDs en vez de autoincrement?

UUIDs se generan sin consultar la BD (sin secuencia). Se pueden crear IDs en el dominio/factory antes de persistir. El número secuencial (`numero`) sí es auto-incremental porque es requisito de negocio visible al usuario.

15. ¿Qué pasa si dos operadores asignan el mismo requerimiento simultáneamente?

Actualmente no hay control de concurrencia. Last-write-wins. En producción se debería usar optimistic locking (verificar versión del documento) o transacciones MongoDB.

16. ¿Por qué el Supervisor no puede comentar?

Regla de negocio del enunciado. Se implementa con polimorfismo: `Supervisor.puede_comentar()` retorna `False`. El `ComentarioService` consulta este método y lanza `PermissionDeniedError`. Además, el router excluye Supervisor de `require_roles`.

17. ¿Cómo verificás la compatibilidad categoría-servicio?

Cada `CategoriaIncidente` define `servicios_compatibles() -> Set[str]`. Ej: `BloqueoSIM.servicios_compatibles()` retorna `{"telefonía"}`. En el router se valida `categoria.es_compatible_con(servicio.nombre)` antes de delegar al service.

18. ¿Cómo se numera cada requerimiento?

`repo.siguiente_numero()` retorna `len(listar_todos()) + 1` (in-memory) o `count_documents({}) + 1` (MongoDB). El service asigna el número después de que la Factory crea la entidad.

19. ¿Qué pasaría si quisieras agregar un estado "EN_ESPERA"?

Creo `EstadoEnEspera(EstadoRequerimiento)` que define qué transiciones permite. Agrego la transición desde un estado existente. Tests existentes siguen pasando. Solo agrego nuevos tests. **OCP**.

20. ¿Por qué los tests no usan MongoDB?

Los 111 tests unitarios usan repos in-memory. Son rápidos (milisegundos), no requieren Docker/MongoDB, y son deterministas. Las requests Bruno sí usan MongoDB (integración end-to-end). La separación dominio/infraestructura permite esta flexibilidad.

4.2 – 10 Preguntas técnicas avanzadas

21. ¿Cómo maneja FastAPI la inyección con Depends?

FastAPI construye un grafo de dependencias DAG. Cuando un endpoint declara `usuario=Depends(get_current_user)`, FastAPI detecta que `get_current_user` necesita `credentials=Depends(HTTPBearer())`, ejecuta primero `HTTPBearer()`, luego `get_current_user(credentials)`, y el resultado se inyecta como `usuario`.

22. ¿Cómo funciona bcrypt internamente?

Genera salt aleatorio de 128 bits, lo concatena con el password y aplica Blowfish en un loop configurable (work factor 12 = 4096 iteraciones). El resultado incluye salt, work factor y hash en un string `$2b$12$...`. Al verificar, extrae el salt del hash almacenado y reaplica.

23. ¿Qué contiene el payload de un JWT?

```
{ "sub": "uuid", "email": "maria@test.com", "rol": "Solicitante", '}
```

Firmado con HMAC-SHA256 usando `JWT_SECRET_KEY`. Stateless: el servidor no almacena sesiones.

24. ¿Por qué `slots=True` en los dataclasses?

`__slots__` reemplaza `__dict__`. Menor memoria (~30% menos), acceso más rápido, previene atributos dinámicos accidentales. Ideal para Value Objects que pueden tener muchas instancias.

25. ¿Cómo convertís estado objeto ↔ string en MongoDB?

```
# Guardar: entidad → string
doc["estado"] = req.estado.nombre() # "ABIERTO"
# Leer: string → objeto
_STR_TO_ESTADO = {"ABIERTO": EstadoAbierto, "ASIGNADO": EstadoAsignado}
req.estado = _STR_TO_ESTADO[doc["estado"]]()
```

26. ¿Cómo reconstruís la herencia de Incidente desde MongoDB?

El campo `doc["tipo"]` discrimina. En `from_mongo()`, si `tipo == "Incidente"` se instancia `Incidente` con urgencia y categoría reconstruidas de strings a Value Objects. Si `tipo == "Solicitud"` se instancia `Solicitud`. Los eventos, comentarios y asignaciones se reconstruyen limpiando los del `__post_init__` y restaurando del documento.

27. ¿Qué pasa si un listener del EventBus lanza excepción?

Se propaga y aborta el flujo. Es un defecto conocido. En producción se debería capturar en el bus, usar bus asíncrono con reintentos, o usar transacciones.

28. ¿Puede un técnico resolver un requerimiento que no tiene asignado?

No. Doble validación: `tecnico.resolver(req)` verifica `self.puede_ver(req)` (que compara `req.asignado_a_id() == self.id`), y `req.resolver()` verifica `self.asignacion_actual.tecnico_id != tecnico_id`. Si no coincide → `DomainRuleError`.

29. ¿Por qué no usás un ORM como SQLAlchemy?

Porque uso MongoDB (documental). PyMongo trabaja con dicts que mapeo manualmente a entidades. Un ODM como MongoEngine contaminaría el dominio (modelos heredan de `Document`). El mapeo manual mantiene el dominio puro.

30. ¿Cómo funciona `_tiene_servicio_activo()` para solicitudes?

Cuenta solicitudes resueltas de alta vs baja para ese servicio: `altas_resueltas > bajas_resueltas` → activo. También verifica suscripciones estáticas. Esto permite que el solicitante acceda al servicio sin suscripción previa si tiene una solicitud de alta resuelta.

4.3 – 5 Preguntas difíciles (para detectar si no entendí)

31. Si te pido que cambies MongoDB por PostgreSQL, ¿cuántos archivos cambiarías?

3 lugares:

1. `app/infrastructure/database.py` – conexión a PostgreSQL.
2. `app/repositories/postgres/` (nuevo directorio) – implementaciones de las interfaces ABC con SQLAlchemy o psycopg2.
3. `app/dependencies.py` – cambiar instanciación de repos Mongo por Postgres.

0 cambios en: dominio, servicios, routers, schemas, auth, frontend, tests unitarios. Los 111 tests siguen pasando.

32. ¿Podrías usar este dominio sin web (CLI)?

Sí. `app/domain/` no importa FastAPI, Pydantic ni MongoDB. Puedo crear un script que instancie repos in-memory, instancie services, y llame `service.crear_incidente(...)` directamente. Sin servidor web. El dominio es Python puro.

33. ¿Dónde está la lógica de "solicitante solo ve sus propios requerimientos"?

En dos lugares:

1. `Router( listar_requerimientos )`: filtra la lista `[r for r in requerimientos if r.autor_id == _usuario.id]` cuando `isinstance(_usuario, Solicitante)`.
2. `Router( obtener_requerimiento )`: verifica `if isinstance(_usuario, Solicitante) and requerimiento.autor_id != _usuario.id` → 403.
3. `Dominio( Solicitante.puede_ver(req) )`: retorna `req.autor_id == self.id` – usado por el `ComentarioService` para validar permisos de comentario.

34. ¿Cómo se persiste un Value Object polimórfico como Urgencia en MongoDB?

En `schemas_mongo.py`:

```
# Guardar
_URG_TO_STR = {UrgenciaCritica: "critica", UrgenciaImportante: "importante"}
doc["urgencia"] = next(v for k, v in _URG_TO_STR.items() if isinstance(v, UrgenciaCritica) or isinstance(v, UrgenciaImportante))

# Leer
_STR_TO_URG = {"critica": UrgenciaCritica, "importante": UrgenciaImportante}
urgencia = _STR_TO_URG[doc["urgencia"]]()
```

Se mapea clase a string para guardar y string a instancia de clase para leer.

35. Si hay 1000 supervisores, ¿qué pasa cuando un operador asigna un técnico?

`notificacion_service.notificar_accion()` busca `usuario_repo.listar_supervisores_de(actor_id)` — solo los supervisores que monitorean a ese operador. Si los 1000 monitorean a ese operador, se crean 1000 notificaciones sincrónicamente. **Es un problema de escalabilidad conocido.** Soluciones:

- EventBus asíncrono (Celery/RabbitMQ)
- Notificaciones lazy (calcular al consultar)
- Limitar monitoreados por supervisor

En este proyecto académico, la carga es baja y priorizo claridad sobre rendimiento.

PARTE 5 – GUION DE EXPOSICIÓN

5.1 Exposición de 7 minutos

Minuto 0-1: Contexto

"El sistema es una mesa de ayuda para la cooperativa Comunicarlos, que provee telefonía, internet y televisión. Permite a solicitantes reportar incidentes y solicitar altas/bajas de servicios, a operadores gestionar y asignar requerimientos, a técnicos resolver y derivar, y a supervisores monitorear la operación con notificaciones en tiempo real."

Minuto 1-2: Arquitectura

"Implementé una arquitectura en capas con separación estricta. El **dominio** es Python puro — no importa FastAPI, MongoDB ni Pydantic. Los **servicios** orquestan casos de uso delegando al dominio. Los **repositorios** abstraen la persistencia con interfaces que tienen dos

implementaciones: in-memory para tests y MongoDB para producción. Los **routers** manejan HTTP y los **schemas** Pydantic validan el contrato de la API. La inyección de dependencias se centraliza en `dependencies.py`, que es el único punto acoplado a implementaciones concretas."

Minuto 2-3: Patrones de diseño

"Aplicué 7 patrones. El **State Pattern** maneja el ciclo de vida del requerimiento con 4 clases de estado inmutables que definen qué transiciones permite cada uno. El **Strategy Pattern** permite cambiar el algoritmo de asignación de técnicos sin modificar el service. La **Factory** centraliza la creación de incidentes y solicitudes con UUID y estado inicial. El **Observer Pattern** con EventBus y NotificacionListener desacopla las notificaciones. El **Repository Pattern** abstrae la persistencia con interfaces ABC. El **Service Layer** encapsula casos de uso testeables independientemente. Y el **Singleton** gestiona la conexión única a MongoDB."

Minuto 3-4: Flujo de un request

"Cuando un solicitante crea un incidente: FastAPI recibe el POST, Pydantic valida el body con `CrearIncidenteRequest` usando patterns regex, `require_roles` verifica que el JWT sea de un Solicitante, el router mapea strings a Value Objects inmutables y delega al `CrearRequerimientoService`. El service verifica suscripciones al servicio, usa la Factory para crear el Incidente con `EstadoAbierto`, asigna número secuencial, persiste vía repositorio, notifica operadores vía `NotificacionService`, y el router transforma la entidad a `RequerimientoResponse` para retornar JSON."

Minuto 4-5: Autenticación y seguridad

"La autenticación es JWT stateless con bcrypt para passwords. El login verifica credenciales y genera un token con user_id y rol derivado del polimorfismo de la clase Python. `get_current_user` decodifica el token en cada request protegido. `require_roles` es un closure que parametriza los roles permitidos por endpoint. Los operadores, técnicos y supervisores requieren email corporativo `@comunicarlos.com.ar`, validado en el constructor de cada entidad de dominio."

Minuto 5-6: Testing

"Tengo 111 tests unitarios: 82 de dominio y 29 de servicios. Todos usan repositorios in-memory — no necesitan Docker ni MongoDB. Testean estados, transiciones, estrategias, factory, event bus, value objects, permisos y cada caso de uso de los servicios. Además hay requests Bruno

para tests de integración end-to-end. El frontend es una SPA con Bootstrap 5 y JavaScript ES6 vanilla con hash router."

Minuto 6-7: SOLID y conclusión

"Cada clase tiene una única responsabilidad. Los patrones permiten extensión sin modificación. Las interfaces permiten sustituir implementaciones. El dominio no depende de infraestructura. Si cambio MongoDB por PostgreSQL cambio 3 archivos, si cambio FastAPI por Django cambio routers y schemas. El dominio y sus 82 tests quedan intactos. Las debilidades que reconozco: el EventBus es sincrónico, no hay transacciones MongoDB, y la creación de usuarios no usa service dedicado."

5.2 Exposición de 1 minuto

"Desarrollé un sistema de mesa de ayuda con FastAPI, MongoDB y frontend SPA en Bootstrap. La arquitectura es en capas: dominio puro Python, servicios que orquestan casos de uso, repositorios que abstraen MongoDB, y routers FastAPI con schemas Pydantic. Apliqué 7 patrones: State para ciclo de vida del requerimiento, Strategy para asignación, Factory para creación, Observer para notificaciones, Repository, Service Layer y Singleton. La autenticación es JWT con bcrypt. Tengo 111 tests unitarios sin base de datos. Si cambio MongoDB, el dominio no se entera."

5.3 Errores que NO debo cometer al explicar

✗ Errores graves

1. **Decir "el router maneja la lógica de negocio"** → El router delega a services. La excepción son creación de usuarios y login (casos simples sin service dedicado).
2. **Confundir Schema con Entidad** → Schema es DTO para HTTP. Entidad tiene lógica de negocio.
`RequerimientoResponse.estado` es `str`. `Requerimiento.estado` es `EstadoRequerimiento` con métodos.
3. **Decir "el dominio importa FastAPI"** → NUNCA. El dominio es Python puro. Si digo esto, evidencia que no entendí la arquitectura.
4. **Confundir State Pattern con if/elif** → No "uso ifs para verificar el estado". El State Pattern usa polimorfismo: le pregunto al propio objeto estado qué permite.

5. **Decir "el repositorio valida reglas de negocio"** → El repositorio solo persiste y recupera. Las reglas están en el dominio.
6. **No saber qué es frozen** → `frozen=True` hace la instancia inmutable a nivel del lenguaje. Enforcement de Python, no convención.
7. **No saber el flujo de excepción** → Dominio lanza `DomainRuleError` → Service propaga → Router captura → `HTTPException` → JSON `{"detail": "..."} .`
8. **Decir que los tests necesitan MongoDB** → Los 111 tests unitarios usan repos in-memory. Solo Bruno usa MongoDB.

Errores sutiles

9. **No diferenciar validación Pydantic de validación de dominio** → Pydantic: formato (¿es UUID?). Dominio: reglas (¿puede asignar en este estado?).
10. **Olvidar que `require_roles` es un closure** → Es función que retorna función. No una dependencia directa.
11. **Decir que `EstadoAsignado.permite_asignar()` retorna True** → Retorna **False**. Para cambiar técnico se usa `derivar()` , no `asignar()` . `EstadoReabierto` sí permite ambas.
12. **No saber cómo se reconstruye la herencia desde MongoDB** → Discriminador `"tipo"` en el documento, mapeo a clase en `schemas_mongo.py` .
13. **Decir "uso Singleton para todo"** → Solo para conexión MongoDB. Mal aplicado sería antipatrón.
14. **No reconocer debilidades** → EventBus sincrónico, creación de usuarios sin service, no hay transacciones, no hay caching. Reconocer esto muestra madurez técnica.

DOCUMENTO TÉCNICO DE DEFENSA ORAL –

PARTE 2

Sistema de Mesa de Ayuda · Cooperativa Comunicarlos

PARTE 6 – TABLA COMPLETA DE ENDPOINTS

6.1 Endpoints de Health y Auth

#	Método	Ruta	Auth	Roles permitidos	Schema Request	Schema Response	Status OK	Acción principal
1	GET	/health	No	Público	—	dict	200	Health check
2	POST	/auth/login	No	Público	LoginRequest	TokenResponse	200	Genera JWT
3	GET	/auth/me	Sí	Cualquier autenticado	—	UsuarioAutenticadoResponse	200	Info del token

6.2 Endpoints de Usuarios

#	Método	Ruta	Auth	Roles permitidos	Schema Request	Schema Response	Status OK	Acción principal
4	POST	/usuarios/solicita antes	No	Público	CrearSolicitanteRequest	Solicitantereponse	201	Registro público
5	POST	/usuarios/operadores	Sí	Supervisor	CrearOperadorRequest	UsuarioResponse	201	Crear operador + auto-monitoreo

#	Método	Ruta	Auth	Roles permitidos	Schema Request	Schema Response	Status OK	Acción principal
6	POST	/usuarios/tecnicos	Sí	Supervisor	CrearTecnicoRequest	TecnicoResponse	201	Crear técnico + auto-monitoreo
7	POST	/usuarios/supervisores	Sí	Supervisor	CrearSupervisorRequest	SupervisorResponse	201	Crear supervisor
8	GET	/usuarios	Sí	Supervisor	—	List[UsuarioResponse]	200	Listar todos
9	GET	/usuarios/tecnicos	Sí	Operador, Supervisor, Tecnico	—	List[TecnicoResponse]	200	Listar técnicos
10	GET	/usuarios/{id}	Sí	Cualquier autenticado	—	UsuarioResponse	200	Obtener usuario (filtrado por rol)
11	POST	/usuarios/me/suscripciones	Sí	Solicitante	AgregarSuscripcionRequest	SolicitudanteResponse	200	Suscribirse a servicio
12	POST	/usuarios/me/monitorear	Sí	Supervisor	MonitorearEmpleadoRequest	SupervisorResponse	200	Monitorear empleado

6.3 Endpoints de Requerimientos

#	Método	Ruta	Auth	Roles permitidos	Schema Request	Schema Response	Status OK	Service	Patrón involucrado
13	POST	/requerimientos/incidentes	Sí	Solicitante	CrearIncidenteRequest	RequerimientoResponse	201	CrearRequerimientoService	Factory, Value Objects

#	Método	Ruta	Auth	Roles permitidos	Schema Request	Schema Response	Status OK	Service	Patrón involucrado
14	POST	/requerimientos/solicitudes	Sí	Solicitante	Crear SolicitudRequest	RequerimientoResponse	201	Crear RequerimientoService	Factory, Value Objects
15	GET	/requerimientos	Sí	Cualquier autenticado	—	List[RequerimientoResponse]	200	—	Filtrado por rol
16	GET	/requerimientos/{id}	Sí	Cualquier autenticado	—	RequerimientoResponse	200	—	Filtrado por rol
17	POST	/requerimientos/{id}/asignar	Sí	Operador	AsignarTecnicoRequest	RequerimientoResponse	200	AsignarTecnicoService	State Pattern
18	POST	/requerimientos/{id}/derivar	Sí	Tecnico	DerivarRequerimientoRequest	RequerimientoResponse	200	DerivarRequerimientoService	Derivación técnico-a-técnico

#	Método	Ruta	Auth	Roles permitidos	Schema Request	Schema Response	Status OK	Service	Patrón involucrado
19	POST	/requerimientos/{id}/comentarios	Sí	Solicitante, Operador, Tecnico	AgregarComentarioRequest	RequerimientoResponse	200	ComentarioService	Polimorfismo (puede_comentar)
20	POST	/requerimientos/{id}/resolver	Sí	Tecnico	ResolverRequerimientoRequest	RequerimientoResponse	200	ResolverRequerimientoService	State Pattern
21	POST	/requerimientos/{id}/reabrir	Sí	Operador, Tecnico	ReabrirRequerimientoRequest	RequerimientoResponse	200	ReabrirRequerimientoService	State Pattern

6.4 Endpoints de Notificaciones

#	Método	Ruta	Auth	Roles permitidos	Schema Response	Status OK	Acción
22	GET	/notificaciones	Sí	Supervisor, Operador, Tecnico	List[NotificacionResponse]	200	Listar todas
23	GET	/notificaciones/no-leídas	Sí	Supervisor, Operador, Tecnico	List[NotificacionResponse]	200	Solo no leídas
24	GET	/notificaciones/contador	Sí	Supervisor, Operador, Tecnico	ContadorResponse	200	Cantidad no leídas

#	Método	Ruta	Auth	Roles permitidos	Schema Response	Status OK	Acción
25	PATCH	/notificaciones/{id}/leer	Sí	Supervisor, Operador, Tecnico	NotificacionResponse	200	Marcar leída
26	PATCH	/notificaciones/leer-todas	Sí	Supervisor, Operador, Tecnico	MensajeResponse	200	Marcar todas leídas

6.5 Endpoints de Servicios

#	Método	Ruta	Auth	Schema Response	Status OK	Acción
27	GET	/servicios	No	List[ServicioCatalogoResponse]	200	Catálogo público (3 servicios)

Total: 27 endpoints (3 públicos, 24 autenticados)

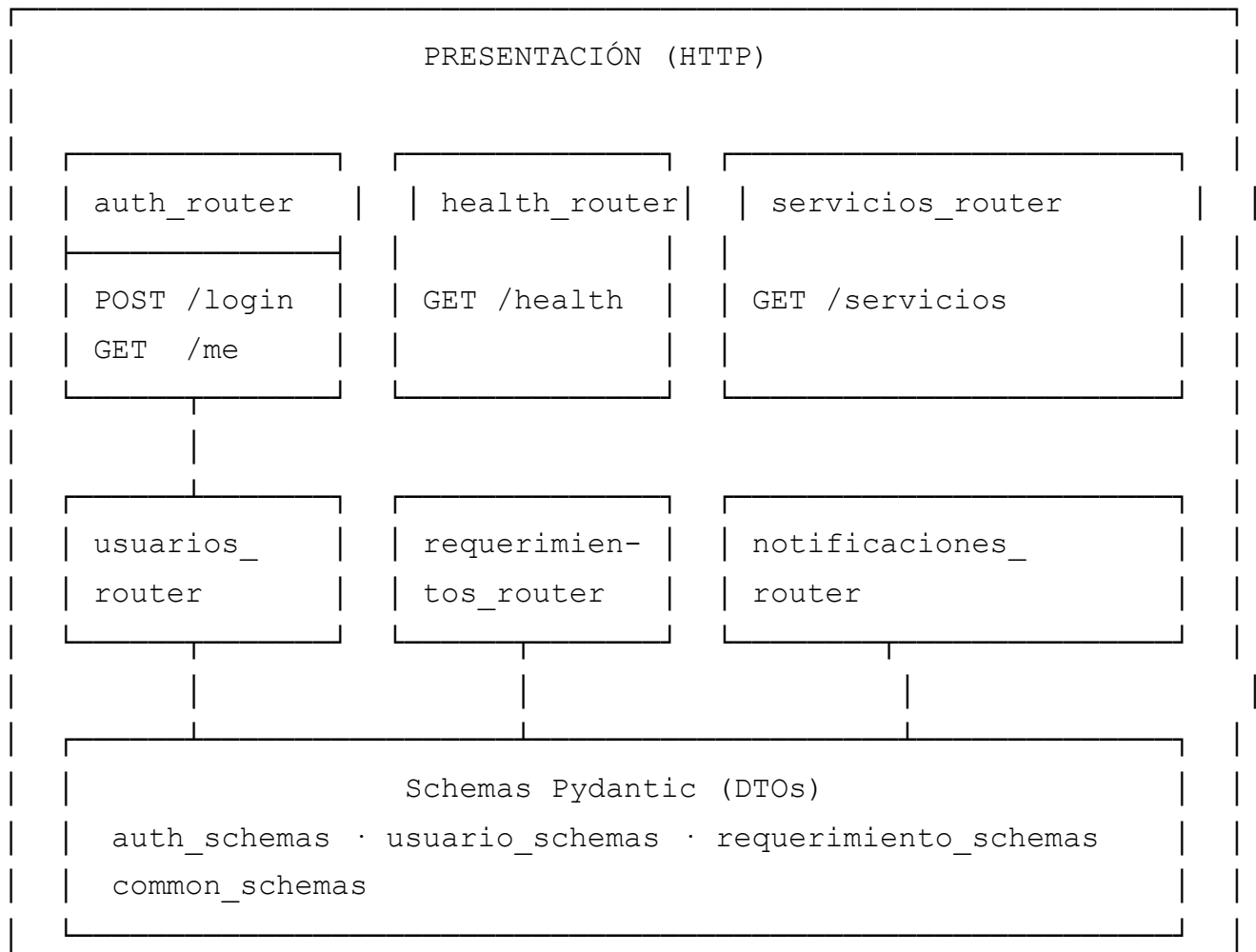
6.6 Matriz de permisos por rol

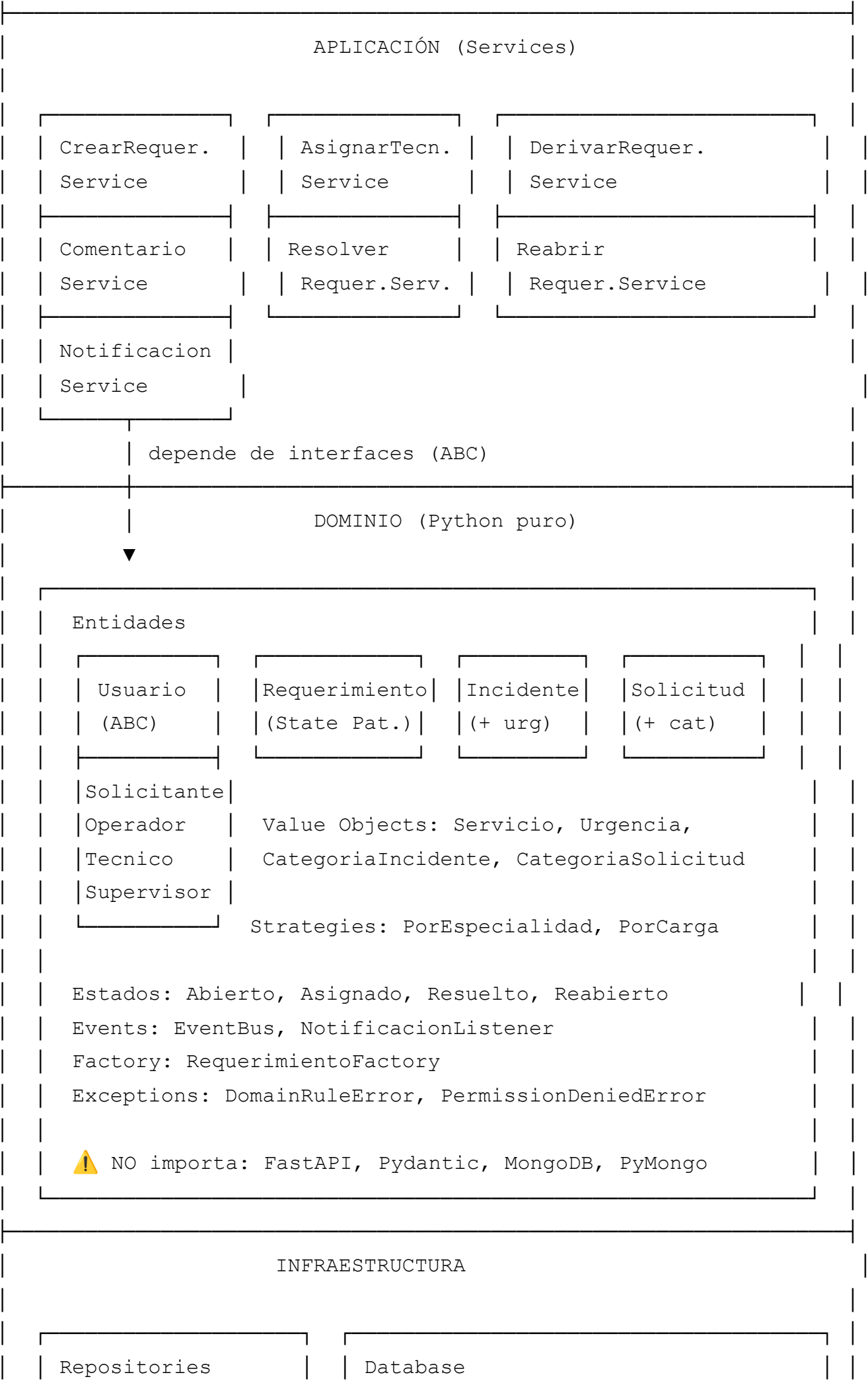
Acción	Solicitante	Operador	Técnico	Supervisor
Registrarse	✓ (público)	✗	✗	✗
Login	✓	✓	✓	✓
Crear operador/técnico/supervisor	✗	✗	✗	✓
Ver usuarios (todos)	✗	✗	✗	✓
Ver técnicos	✗	✓	✓	✓
Ver usuario por ID	Solo sí mismo	Cualquiera	Solo sí mismo	Cualquiera
Suscribirse servicio	✓	✗	✗	✗
Monitorear empleado	✗	✗	✗	✓
Crear incidente/solicitud	✓	✗	✗	✗
Listar requerimientos	Solo propios	Todos	Solo asignados	Todos
Ver detalle requerimiento	Solo propios	Todos	Solo asignados	Todos
Asignar técnico	✗	✓	✗	✗

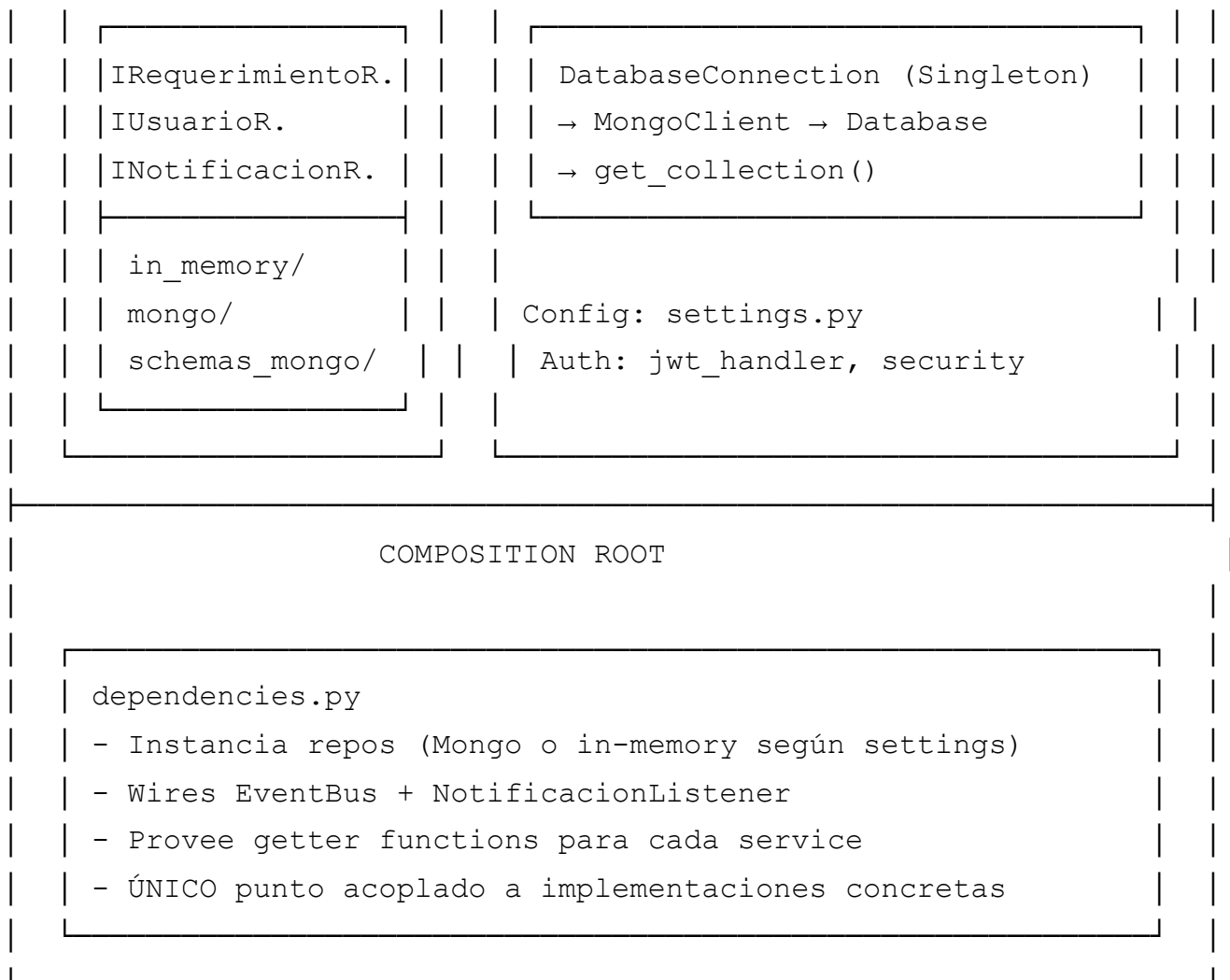
Acción	Solicitante	Operador	Técnico	Supervisor
Derivar requerimiento	✗	✗	✓ (si asignado)	✗
Comentar	Solo propios	Todos	Solo asignados	✗ (regla)
Resolver	✗	✗	✓ (si asignado)	✗
Reabrir	✗	✓	✓	✗
Ver notificaciones	✗	✓	✓	✓
Marcar notificación leída	✗	✓	✓	✓

PARTE 7 – DIAGRAMAS DE DEPENDENCIA Y FLUJO

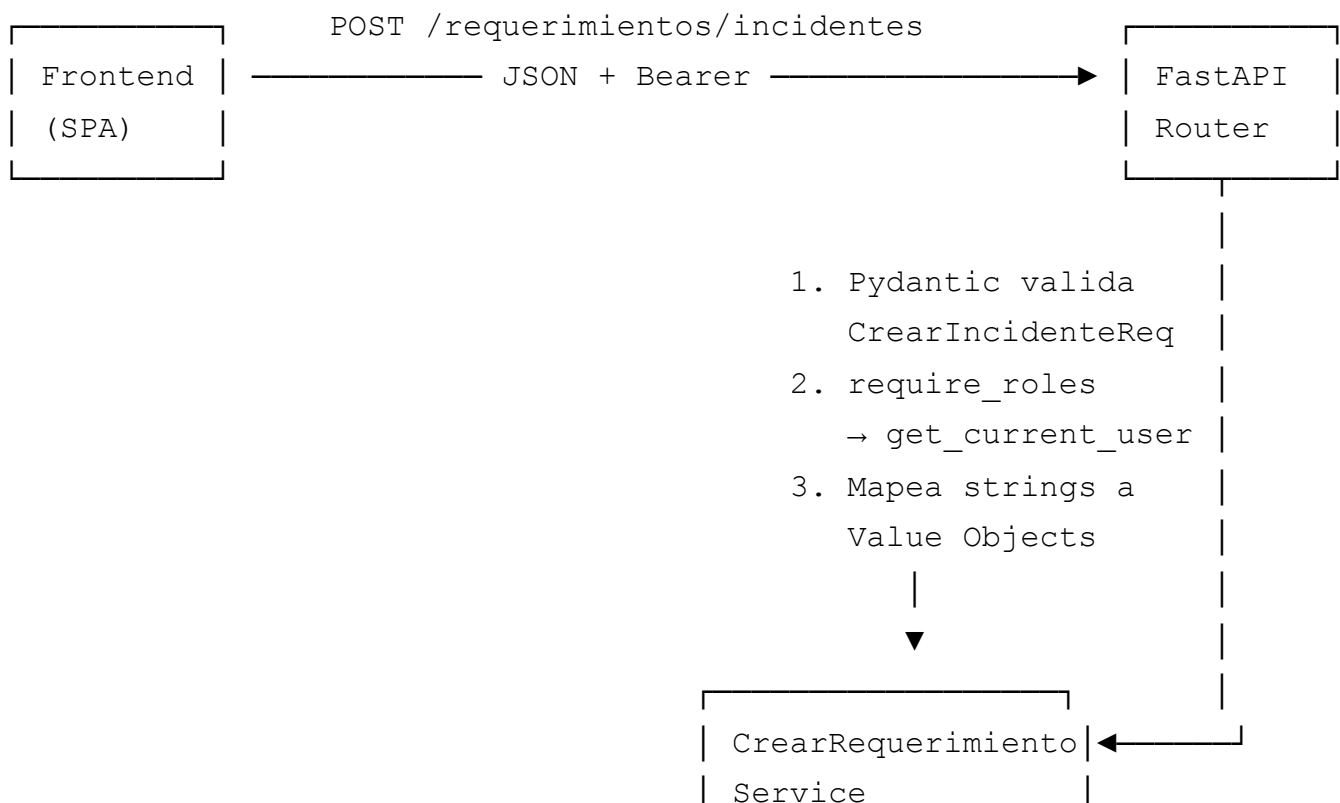
7.1 Diagrama de capas

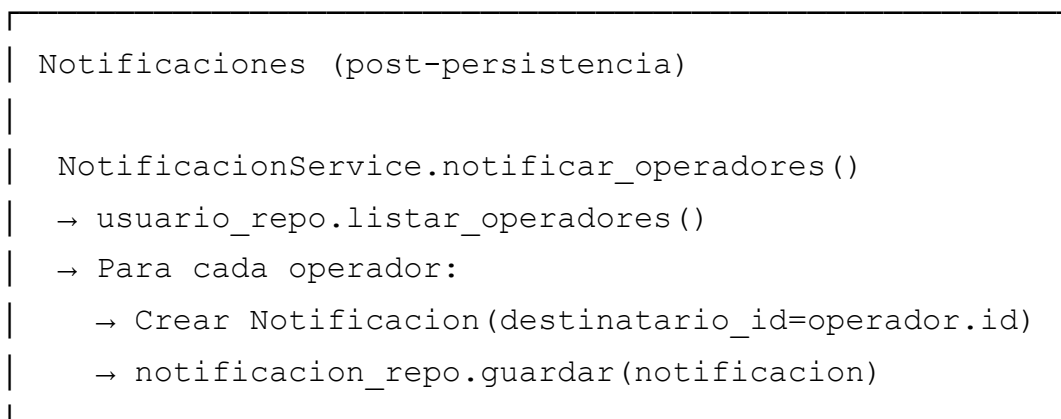
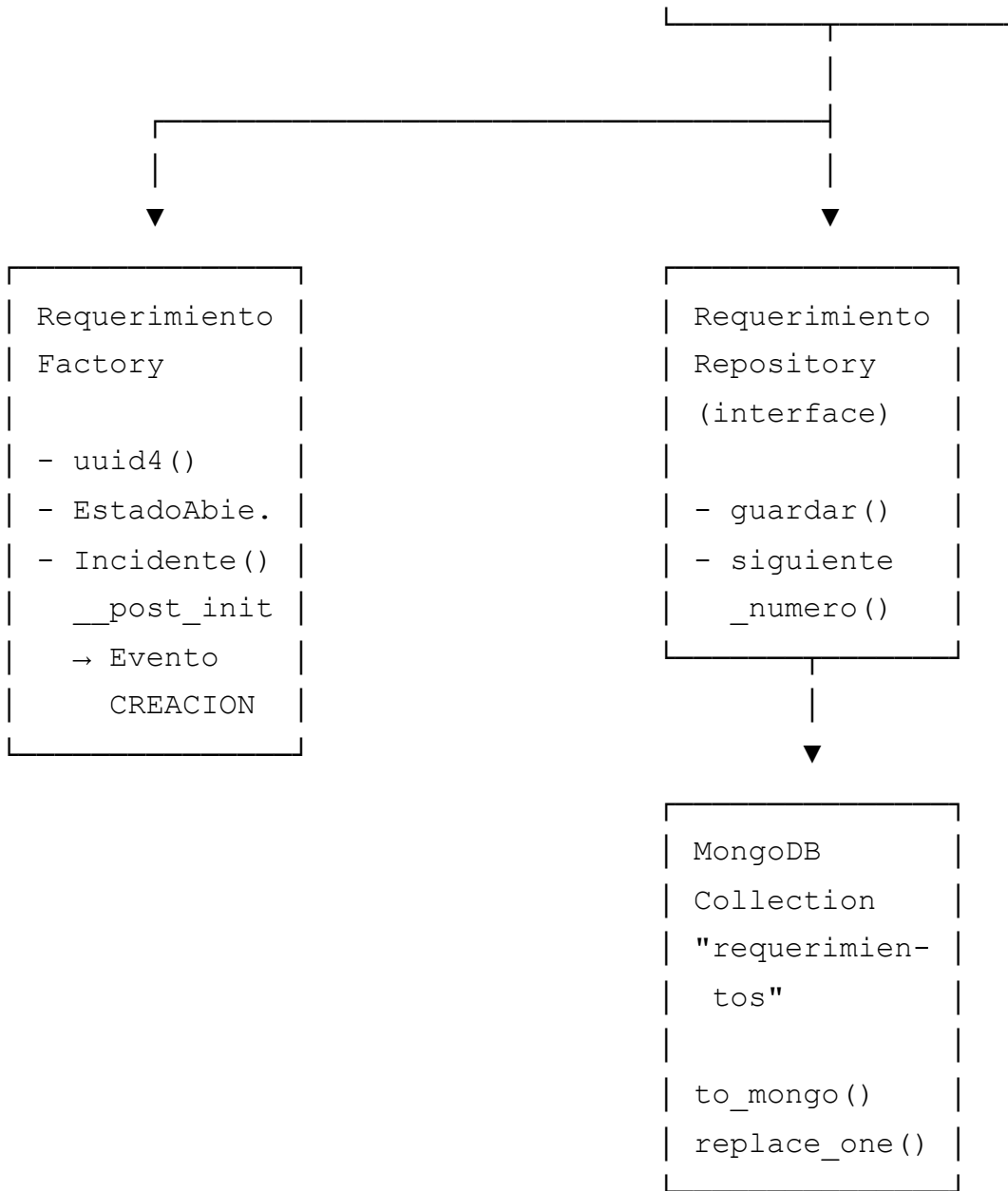






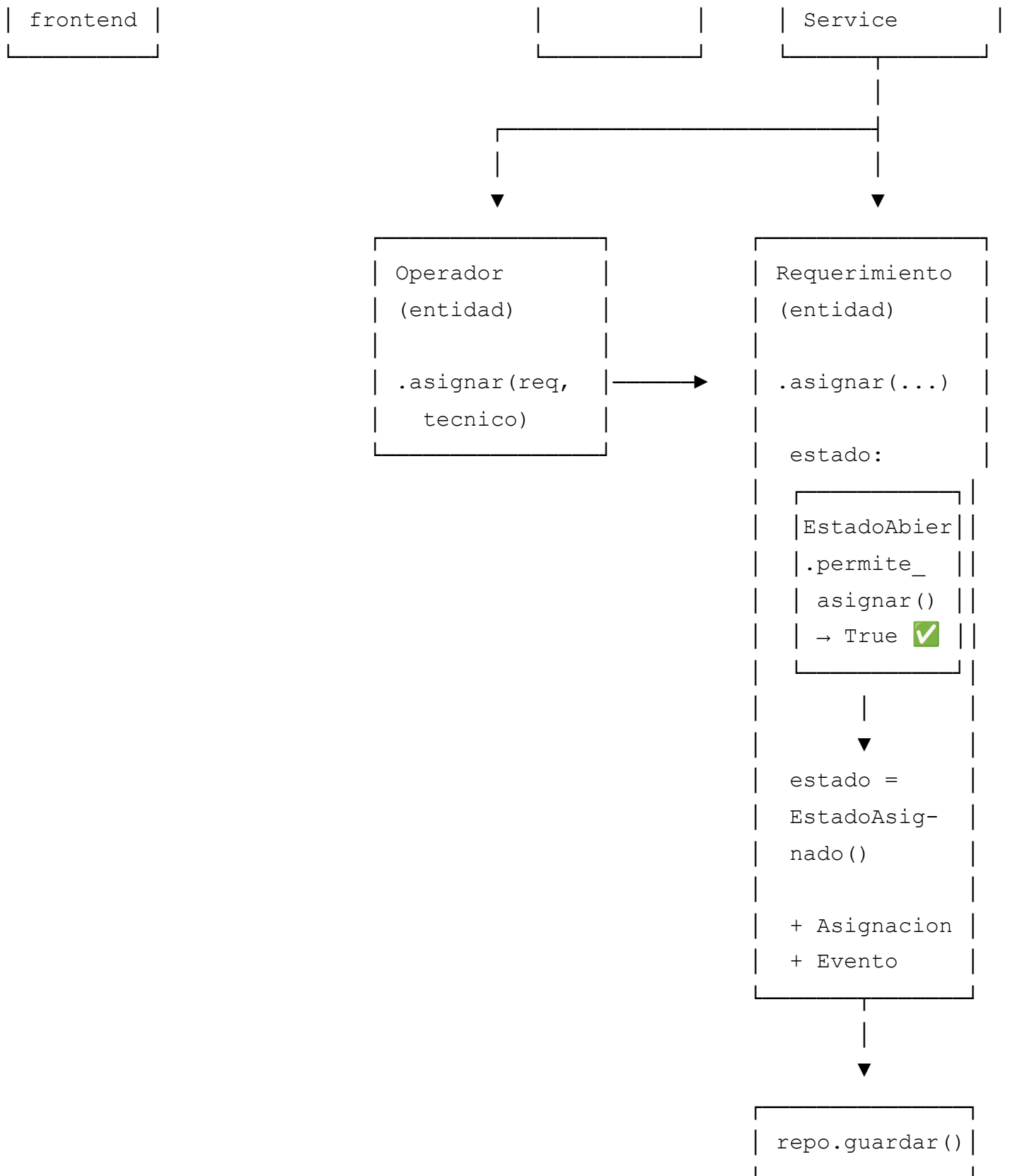
7.2 Flujo de datos: Crear Incidente





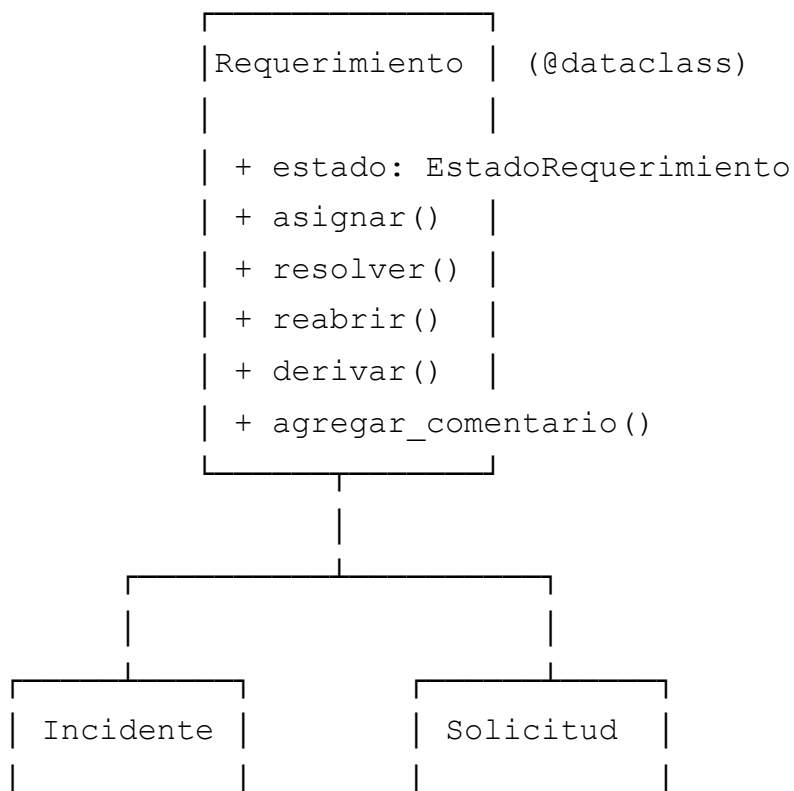
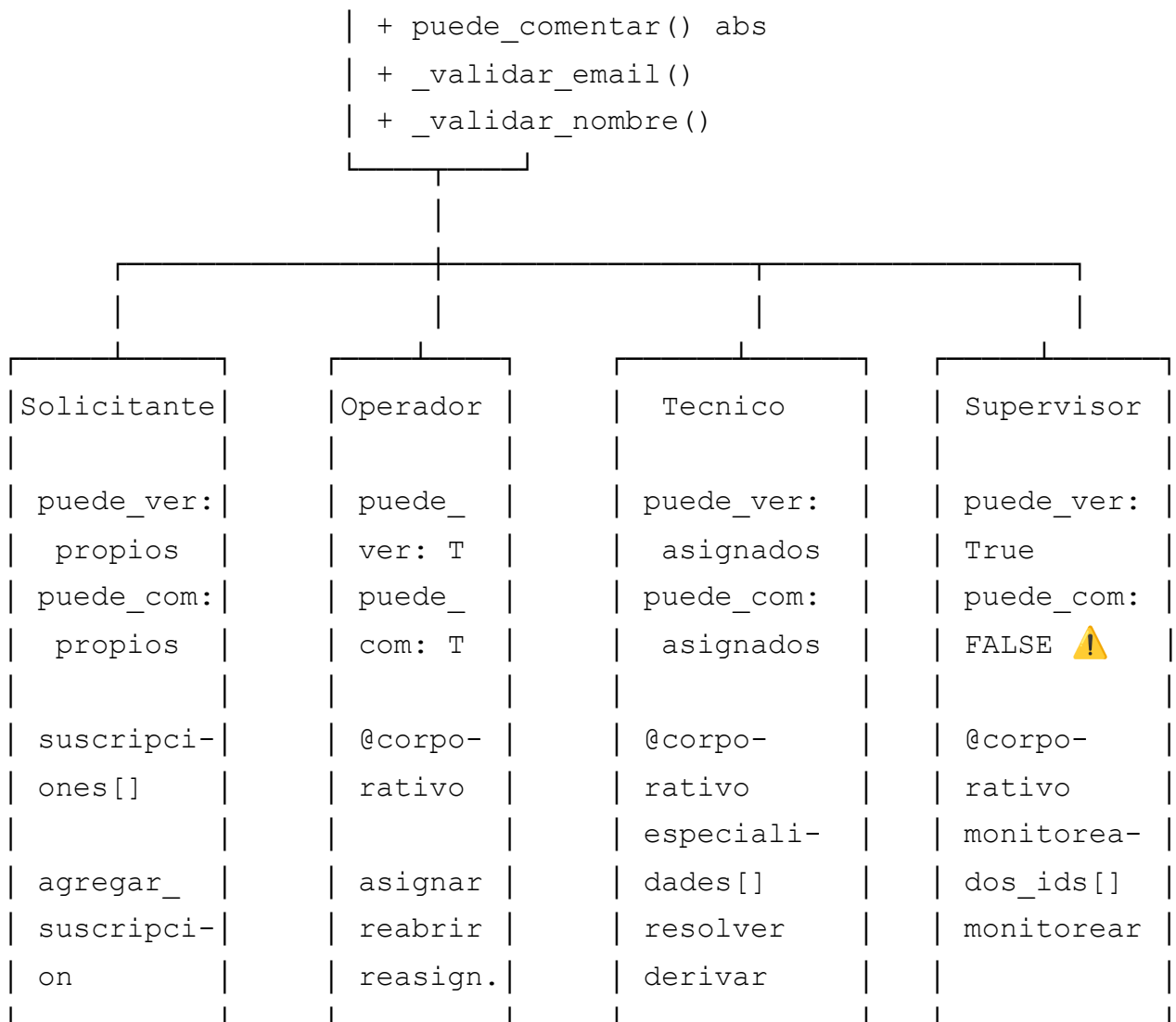
7.3 Flujo de datos: Asignar Técnico con State Pattern



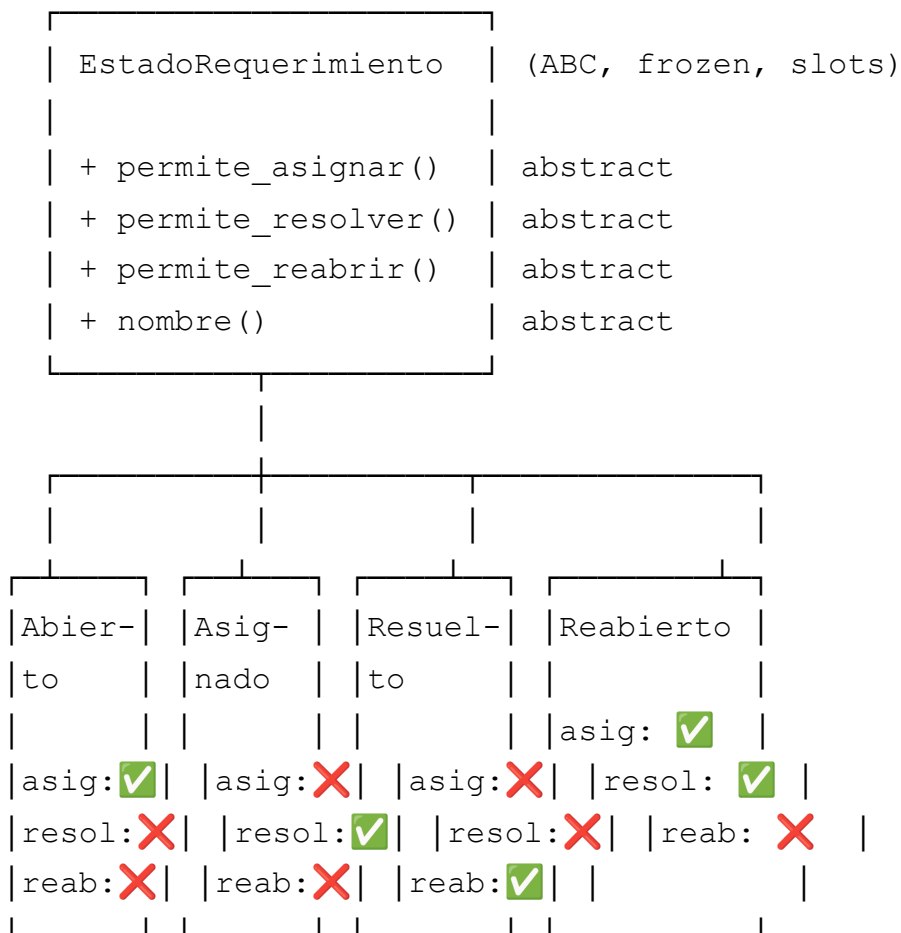


7.4 Diagrama de herencia del dominio

```
class Usuario {
    (ABC, @dataclass)
    + puede_ver() abs
}
```

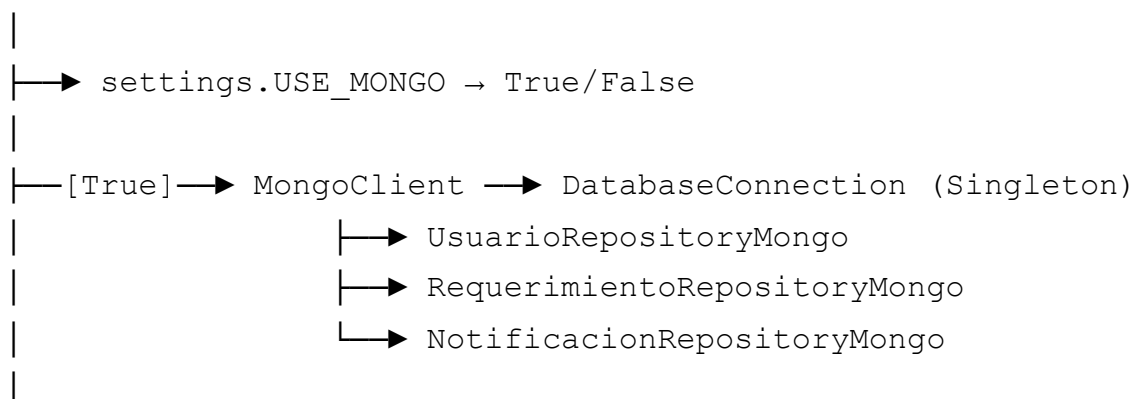


urgencia:	categoria:
Urgencia	CategoriaSo
categoria:	licitud
Categori-	
aIncident	



7.5 Diagrama de dependencias: Inyección

dependencies.py (Composition Root)

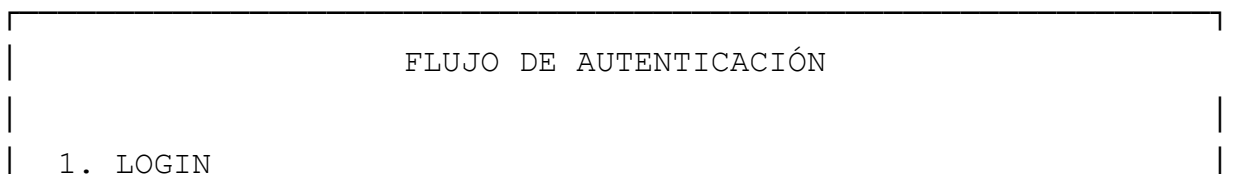


```

└─[False]→ UsuarioRepositoryInMemory
      RequerimientoRepositoryInMemory
      NotificacionRepositoryInMemory
└─
└─→ RequerimientoFactory
└─
└─→ EventBus
      └─→ NotificacionListener
            └─→ callback: notificacion_repo.guardar
└─
└─→ NotificacionService(usuario_repo, notificacion_repo)
└─
└─→ get_crear_requerimiento_service()
      └─→ CrearRequerimientoService(req_repo, usr_repo, factory,
notif_service)
└─
└─→ get_asignar_tecnico_service()
      └─→ AsignarTecnicoService(req_repo, usr_repo, notif_service)
└─
└─→ get_derivar_requerimiento_service()
      └─→ DerivarRequerimientoService(req_repo, usr_repo,
notif_service)
└─
└─→ get_comentario_service()
      └─→ ComentarioService(req_repo, usr_repo, notif_service)
└─
└─→ get_resolver_requerimiento_service()
      └─→ ResolverRequerimientoService(req_repo, usr_repo,
notif_service)
└─
└─→ get_reabrir_requerimiento_service()
      └─→ ReabrirRequerimientoService(req_repo, usr_repo,
notif_service)

```

7.6 Flujo de autenticación JWT



POST /auth/login {email, password}

|

▼

repo.obtener_por_email(email) → Usuario

verify_password(password, usuario.password_hash) → bool

|

▼ (si ok)

create_access_token(user_id, email, rol)

|

▼

JWT = header.payload.signature (HS256)

payload = {sub: uuid, email, rol, exp}

|

▼

Response: {access_token: "eyJ...", token_type: "bearer"}

2. REQUESTS AUTENTICADOS

GET /requerimientos

Header: Authorization: Bearer eyJ...

|

▼

HTTPBearer() extrae token

|

▼

decode_access_token(token)

→ Verifica firma (JWT_SECRET_KEY)

→ Verifica exp (no expirado)

→ payload = {sub, email, rol}

|

▼

repo.obtener_por_id(UUID(payload["sub"]))

→ Retorna entidad de dominio (Solicitante, etc.)

|

▼

require_roles(["Operador"]):

usuario.__class__.__name__ in ["Operador"]

→ True → continúa

→ False → HTTPException(403)

3. STATELESS

- No hay sesiones del lado del servidor

- Cada request trae su token
- El token contiene toda la info necesaria
- Expira en `JWT_EXPIRE_MINUTES` (default 60)

PARTE 8 – CHECKLIST Y MATERIAL COMPLEMENTARIO

8.1 Checklist de defensa

Antes de presentar

- ☐ Docker compose `up` funciona y crea supervisor automáticamente
- ☐ Swagger UI accesible en `/docs`
- ☐ ReDoc accesible en `/redoc`
- ☐ Frontend SPA funcional en `/app`
- ☐ Puedo hacer login con `admin@comunicarlos.com.ar`
- ☐ Puedo crear un solicitante, suscribirlo, crear incidente
- ☐ Puedo asignar técnico, derivar, comentar, resolver, reabrir
- ☐ Puedo ver notificaciones del supervisor
- ☐ Los 111 tests pasan: `python -m pytest tests/ -v`

Durante la defensa

- ☐ Nombro la arquitectura en capas y su propósito
 - ☐ Explico al menos 3 patrones con ejemplo de código
 - ☐ Muestro flujo de un request completo (ej: crear incidente)
 - ☐ Muestro State Pattern con transición de estado
 - ☐ Menciono inyección de dependencias y cómo se cablea
 - ☐ Menciono que el dominio es Python puro (sin dependencias externas)
 - ☐ Muestro tests corriendo (sin MongoDB)
 - ☐ Reconozco al menos 2 debilidades (EventBus sincrónico, sin transacciones)
-

8.2 Resumen ejecutivo – 1 página

Proyecto

Mesa de ayuda para cooperativa Comunicarlos. Gestión de incidentes y solicitudes de servicios de telefonía, internet y TV.

Stack

Componente	Tecnología
Backend	Python 3.11+, FastAPI
Base de datos	MongoDB (PyMongo)
Autenticación	JWT (python-jose) + bcrypt (passlib)
Frontend	HTML5 + Bootstrap 5.3 + JavaScript ES6 (SPA hash router)
Container	Docker + Docker Compose
Testing	pytest (111 tests unitarios)
Documentación	Sphinx auto-generada

Arquitectura

4 capas con dependencia descendente:

- 1. **Presentación** (Routers + Schemas Pydantic)
- 2. **Aplicación** (7 Services)
- 3. **Dominio** (Entidades, Value Objects, States, Strategies, Factory, Events)
- 4. **Infraestructura** (Repositories Mongo/InMemory, Database Singleton)

Patrones aplicados

Patrón	Dónde	Para qué
State	<code>EstadoRequerimiento</code> (4 clases)	Ciclo de vida del requerimiento
Strategy	<code>EstrategiaAsignacion</code> (2 impl.)	Algoritmo de asignación configurable
Factory	<code>RequerimientoFactory</code>	Creación estandarizada de Incidente/Solicitud
Observer	<code>EventBus</code> + <code>NotificacionListener</code>	Notificaciones desacopladas
Repository	3 interfaces × 2 implementaciones	Persistencia intercambiable

Patrón	Dónde	Para qué
Service Layer	7 servicios	Casos de uso orquestados
Singleton	<code>DatabaseConnection</code>	Conexión única a MongoDB

Números clave

Métrica	Valor
Entidades de dominio	11 (Usuario×4 + Requerimiento×3 + Asignacion + Comentario + Evento + Notificacion + Suscripcion)
Value Objects	10 (Servicio, 3 Urgencias, 3 Categorías Incidente, 2 Categorías Solicitud, Servicio catálogo)
Endpoints API	27 (3 públicos, 24 autenticados)
Tests unitarios	111 (82 dominio + 29 servicios)
Archivos Python (app/)	~50
Patrones de diseño	7

8.3 Glosario técnico

Término	Definición en este proyecto
Entidad	Objeto con identidad (UUID) y ciclo de vida (mutable). Ej: <code>Requerimiento</code> , <code>Usuario</code>
Value Object	Objeto inmutable (<code>frozen=True</code>) sin identidad propia, definido por sus atributos. Ej: <code>Servicio</code> , <code>UrgenciaCritica</code>
Aggregate	Cluster de entidades con una raíz. <code>Requerimiento</code> es raíz de Comentarios, Eventos, Asignaciones
Repository	Abstracción (ABC) para persistencia. Oculta si es in-memory o MongoDB
Service	Orquestador de un caso de uso. Coordina entidades y repositorios
Factory	Creador centralizado de entidades complejas. Asigna UUID y estado inicial
DTO	Data Transfer Object – los schemas Pydantic que cruzan la frontera HTTP
Composition Root	<code>dependencies.py</code> – único punto donde se instancian implementaciones concretas

Término	Definición en este proyecto
Closure	Función que retorna función, capturando variables del scope exterior. <code>require_roles()</code>
Discriminador	Campo <code>tipo</code> en MongoDB que indica qué subclase instanciar al leer
State Pattern	El objeto delega comportamiento a su estado actual (un objeto separado)
Strategy Pattern	Algoritmo intercambiable en runtime sin cambiar el cliente
Observer Pattern	Suscriptores notificados cuando ocurre un evento, sin acoplamiento directo
Singleton	Garantiza una única instancia de una clase
DIP	Dependency Inversion Principle – depender de abstracciones, no de implementaciones
OCP	Open/Closed Principle – abierto a extensión, cerrado a modificación
SRP	Single Responsibility Principle – cada clase tiene una única razón de cambio
LSP	Liskov Substitution Principle – subtipos sustituibles donde se espera el tipo base

8.4 Archivos del proyecto – Mapa rápido

```

app/
├── main.py                # Entry point, routers, CORS,
bootstrap supervisor
├── dependencies.py        # Composition Root, inyección de
dependencias
|
├── auth/
|   ├── auth_dependencies.py    # get_current_user, require_roles
(closures)
|   ├── jwt_handler.py         # create_access_token,
decode_access_token (HS256)
|   └── security.py            # hash_password, verify_password
(bcrypt)
|
├── config/
|   └── settings.py            # MONGO_URI, JWT_SECRET, USE_MONGO,
etc.
|

```

```

└─ domain/
FastAPI/MongoDB
|   └─ entities/
|       └─ usuario.py
abstractos
|       └─ solicitante.py
ve propios
|       └─ operador.py
asigna/reabrir
|       └─ tecnico.py
resolv/deriv
|       └─ supervisor.py
comenta
|       └─ requerimiento.py
via properties
|       └─ incidente.py
cat. incidente
|       └─ solicitud.py
solicitud
|       └─ estado_requerimiento.py
Abierto/Asignado/Resuelto/Reab.
|       └─ asignacion.py
|       └─ comentario.py
|       └─ evento.py
|       └─ notificacion.py
|       └─ suscripcion.py
|
|   └─ value_objects/
|       └─ servicio.py
fijos
|       └─ urgencia.py
Menor(1)
|       └─ categoria_incidente.py
PerdidaEquipo
|       └─ categoria_solicitud.py
|
|   └─ strategies/
|       └─ estrategia_asignacion.py
|       └─ estrategia_por_especialidad.py
|       └─ estrategia_por_carga.py
injectable)

```

⚠ PYTHON PURO – sin imports de

ABC con puede_ver/puede_comentar

Hereda Usuario, suscripciones, solo

Hereda Usuario, email corporativo,

Hereda Usuario, especialidades,

Hereda Usuario, monitorea, NO

State Pattern, historial inmutable

Hereda Requerimiento + urgencia +

Hereda Requerimiento + cat.

4 estados frozen:

Mutable, cerrar(), esta_activa()

Frozen – inmutable una vez creado

Frozen – hecho histórico

Mutable – marcar_leida()

Frozen – snapshot

Catálogo 3 servicios con UUIDs

ABC → Critica(3), Importante(2),

ServicioInaccesible, BloqueoSIM,

AltaServicio, BajaServicio

ABC: elegir_tecnico()

Primera coincidencia

min() por carga (callable

```

|   |
|   |─ factories/
|   |   └─ requerimiento_factory.py # crear_incidente(),
crear_solicitud()
|   |
|   |─ events/
|   |   └─ evento_dominio.py          # @dataclass(frozen): tipo, datos,
fecha
|   |   └─ domain_event_listener.py # ABC: handle(evento)
|   |   └─ event_bus.py              # suscribir/publicar → lista de
listeners
|   |   └─ notificacion_listener.py # Crea Notificacion vía callback
|   |
|   └─ exceptions/
|       └─ domain_errors.py          # DomainError → DomainRuleError,
Permission, Validation
|
|─ services/
|   └─ crear_requerimiento_service.py # Factory + suscripción + número
+ notif. operadores
|   └─ asignar_tecnico_service.py     # Directo + Strategy + notif.
supervisores
|   └─ derivar_requerimiento_service.py # Derivación + actor override +
notif.
|   └─ comentario_service.py         # Polimorfismo puede_comentar +
notif.
|   └─ resolver_requerimiento_service.py # State ASIGNADO→RESUELTO +
notif.
|   └─ reabrir_requerimiento_service.py # State RESUELTO→REABIERTO +
notif.
|   └─ notificacion_service.py       #
notificar_accion/operadores/usuario + listar
|
|─ repositories/
|   └─ base.py                       # 3 ABCs: IRequerimientoR, IUsuarioR,
INotificacionR
|   └─ in_memory/                   # Dict[UUID, T] para tests
|       └─ (3 archivos)
|   └─ mongo/                       # PyMongo con schemas_mongo.py
(to/from_mongo)
|       └─ (4 archivos)

```

```

|
├─ routers/
|   └─ health_router.py           # GET /health
|   └─ auth_router.py             # POST /login, GET /me
|   └─ usuarios_router.py         # CRUD usuarios + suscripciones +
monitoreo
|   └─ requerimientos_router.py   # CRUD +
asignar/derivar/comentar/resolver/reabrir
|   └─ notificaciones_router.py   # Listar + marcar leída
|   └─ servicios_router.py       # GET /servicios (público)
|
├─ schemas/
|   └─ auth_schemas.py           # LoginRequest, TokenResponse
|   └─ usuario_schemas.py       # Crear*Request, *Response
|   └─ requerimiento_schemas.py  # Crear*Request, AsignarRequest,
Response
|   └─ common_schemas.py        # ErrorResponse, MensajeResponse
|
└─ infrastructure/
    └─ database.py               # DatabaseConnection Singleton +
get_collection

tests/
├─ domain/    # 82 tests - entidades, estados, estrategias, factory,
events, VOs
└─ services/  # 29 tests - cada service con repos in-memory

```

8.5 Comandos de referencia rápida

```

# Levantar con Docker
docker-compose up --build

# Correr tests
python -m pytest tests/ -v

# Solo tests de dominio
python -m pytest tests/domain/ -v

# Solo tests de servicios
python -m pytest tests/services/ -v

```

```
# Contar tests
python -m pytest tests/ --collect-only -q | tail -1
# → 111 tests collected

# Correr tests con cobertura (si está instalado)
python -m pytest tests/ --cov=app --cov-report=term-missing

# Ver Swagger UI
# http://localhost:8000/docs

# Ver ReDoc
# http://localhost:8000/redoc

# Frontend SPA
# http://localhost:8000/app

# Documentación Sphinx
# http://localhost:8000/documentacion
```

8.6 Respuestas a preguntas sorpresa

"¿Por qué no usaste Django?"

Django es full-stack con ORM propio y template engine. FastAPI es más ligero, async-ready, con validación automática vía Pydantic y documentación OpenAPI auto-generada. Para una API REST con frontend SPA, FastAPI es mejor fit. Además, Django's ORM contaminaría el dominio — los modelos heredarían de `models.Model`.

"¿Esto escala?"

Horizontalmente sí — FastAPI es stateless (JWT, sin sesiones). Se puede replicar detrás de un load balancer. MongoDB escala con replica sets y sharding. Verticalmente hay limitaciones: EventBus sincrónico, no hay caching. En producción se agregaría Redis para cache, Celery para notificaciones asíncronas, y MongoDB transactions.

"¿Cuántas líneas de código?"

~50 archivos Python en `app/`, ~18 archivos de tests. El dominio es la capa más grande por diseño — contiene toda la lógica de negocio.

"¿El frontend está bien hecho?"

Es funcional pero simple. SPA con hash router, Bootstrap 5.3, JavaScript ES6 vanilla. Sin framework como React/Vue. Para este proyecto académico priorizo el backend y la arquitectura.

"¿Por qué tests unitarios y no de integración?"

Los 111 tests unitarios validan reglas de negocio rápido y sin dependencias. Las requests Bruno en el directorio `bruno/` sirven como tests de integración end-to-end. Ambos se complementan: unitarios para lógica aislada, Bruno para verificar el sistema completo con MongoDB.

"¿Qué mejorarías?"

1. **Notificaciones asíncronas** con Celery + RabbitMQ
2. **Transacciones MongoDB** para atomicidad
3. **Service de creación de usuarios** para consistencia
4. **Filtrado en query MongoDB** en vez de traer todo y filtrar en Python
5. **Tests de integración automatizados** con testcontainers
6. **Caching con Redis** para usuarios y configuración
7. **Rate limiting** en endpoints públicos
8. **Logs estructurados** con logging y correlation IDs